

DEEP LEARNING LECTURE SERIES

Deep Learning Architecture for Transformer Networks

From Encoder-Decoder to BERT, GPT, LLMs, and
LVMs

Aditya Nigam, Indian Institute of Technology Mandi

April 18, 2026



Why Transformers? Scope of This Talk

THE TRANSFORMER PARADIGM

Transformers revolutionized deep learning by introducing **parallel sequence modeling** that eliminates the need for recurrence. They have since evolved into a **universal architecture** powering state-of-the-art models in both language and vision domains.



What You Will Learn in This Series

- **Architecture Fundamentals:** Encoder-decoder structure, attention mechanisms, and positional encoding.
- **Language Models:** Bidirectional encoding with BERT and autoregressive generation with GPT.
- **Modern LLM Training:** Pre-training, fine-tuning pipelines, and Reinforcement Learning from Human Feedback (RLHF).
- **Large Vision Models (LVMs):** Extending transformers to computer vision (ViT) and self-supervised methods (DINO).

Agenda

PART 01



Core Architecture

- **Transformer fundamentals:** Overview and intuition
- **Attention mechanisms:** Self, cross, and multi-head attention
- **Block structures:** Encoder and decoder deep dive
- **Positional encoding:** Adding sequence order

PART 02



Language Models

- **BERT & DistilBERT:** Bidirectional encoding, MLM, and NSP
- **GPT variants:** Autoregressive decoding and generation
- **LLM Pipeline:** Pre-training to fine-tuning
- **RLHF:** Reward modeling and reinforcement learning

PART 03



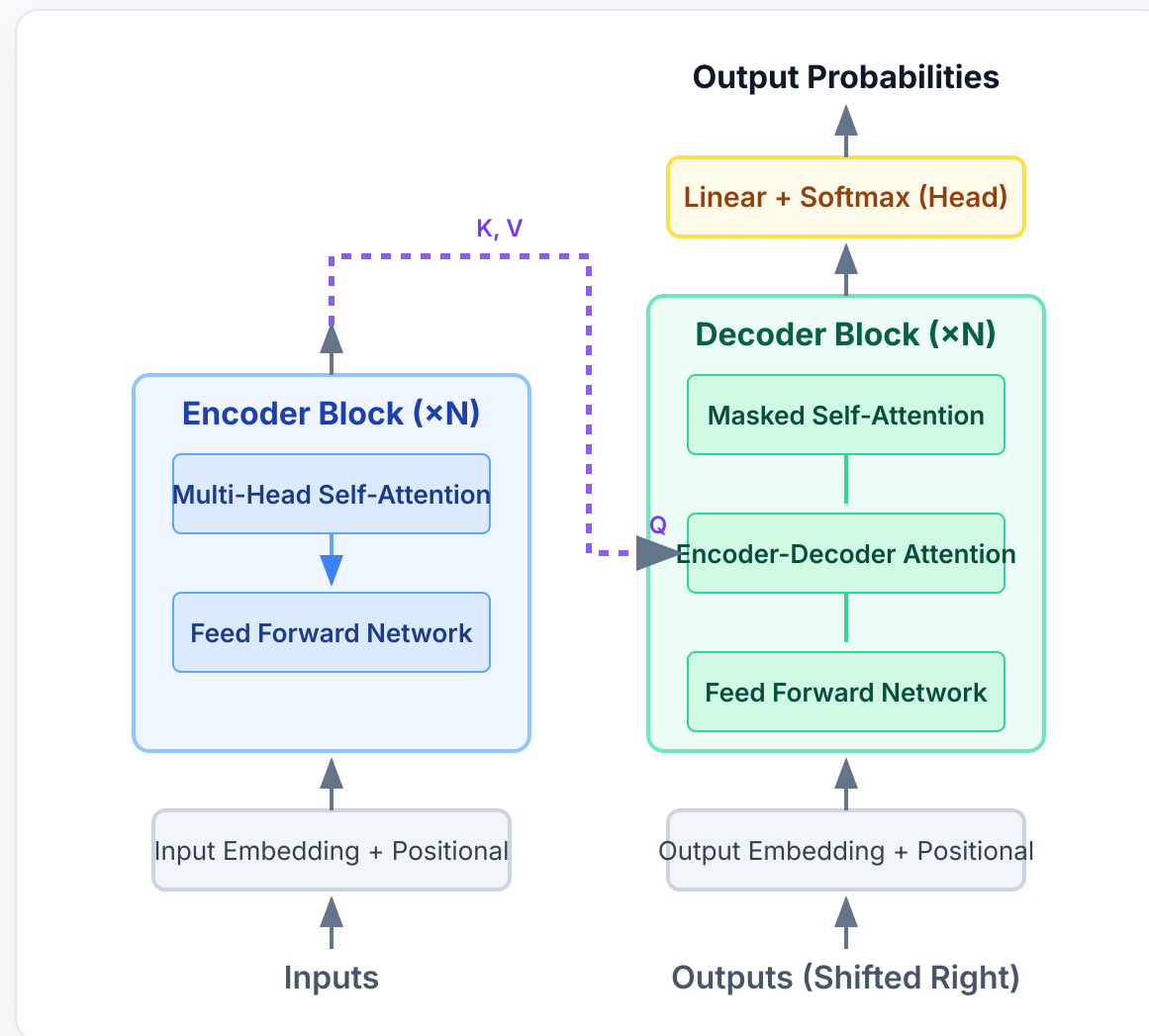
Vision & Beyond

- **LVM Introduction:** Scaling vision models
- **Vision Transformer (ViT):** Adapting self-attention for images
- **DINO framework:** Self-distillation and self-supervised learning
- **Summary:** Key takeaways and future directions

Transformer at a Glance

ARCHITECTURE OVERVIEW

- **Sequence-to-sequence** mapping achieved entirely without Recurrent (RNN) or Convolutional (CNN) networks.
- **Key Building Blocks:**
 - Encoder Stack ($\times N$ layers)
 - Decoder Stack ($\times N$ layers)
 - Predicting / Output Head
- Highly **parallelizable** computation enabled by self-attention mechanisms, significantly reducing training time.
- Forms the foundation for both modern **Large Language Models** (LLMs) and **Large Vision Models** (LVMs).



Encoder-Decoder Architecture

CORE FUNCTIONAL FLOW

- **Primary Framework:** The Encoder-Decoder architecture forms the central backbone of the original Transformer model.
- **The Encoder Block:** Ingests the sequence of input tokens (words) simultaneously and transforms them into a continuous sequence of high-dimensional **latent vectors**.
- **The Decoder Block:** Takes the rich latent representations from the encoder and converts them back into an **output sequence**.
- **Generation Process:** While the encoder processes all inputs in parallel, the decoder typically generates the output sequence one token at a time (autoregressively).

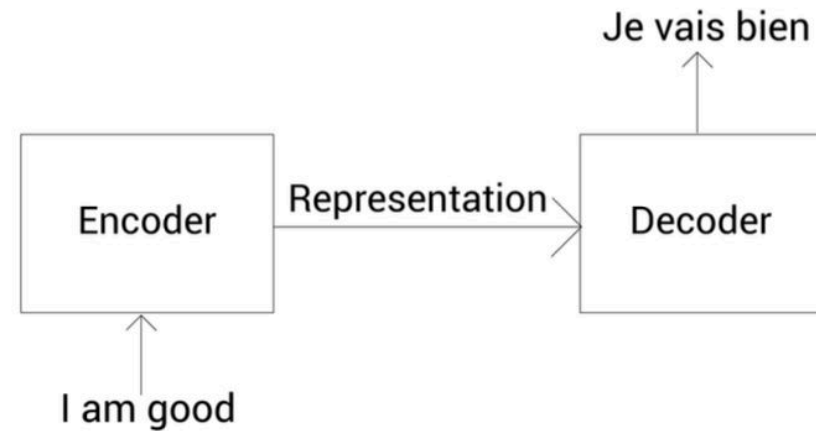
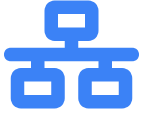


Figure 1.1 – Encoder and decoder of the transformer

Key Components of Transformers



Attention Mechanisms

- **Self-attention:** Enables the model to relate different positions of a single sequence to compute its representation.
- **Cross-attention:** Allows the decoder to attend to the encoded source sequence during generation.
- **Multi-head:** Projects queries, keys, and values into multiple subspaces in parallel.



Positional Encoding

- **Order matters:** Transformers lack recurrence or convolution, making sequence order invisible by default.
- **Injection:** Injects information about relative or absolute positions of tokens directly into the input embeddings.
- **Method:** Uses fixed sinusoidal functions of varying frequencies, or learned parameter vectors.



Normalization & FFN

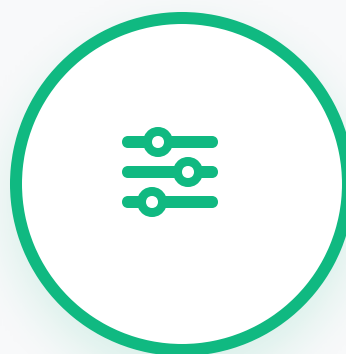
- **Layer Normalization:** Applied across the feature dimension for each token independently. Stabilizes and speeds up training.
- **Residual Connections:** Skip connections around attention and FFN layers prevent vanishing gradients.
- **Feed-Forward Network:** Two dense layers with a non-linearity (ReLU/GELU) applied uniformly to each position.

Training vs Inference Overview



STAGE 01
Pre-training

Training on massive unsupervised corpora to build foundational sequence representations. Highly resource intensive.



STAGE 02
Fine-tuning

Adapting the pre-trained model on smaller, task-specific datasets. A predicting head is typically attached here.



STAGE 03
Inference

Model generates predictions for downstream tasks. Output vectors are tokenized and scored against the vocabulary.

Full Transformer Architecture

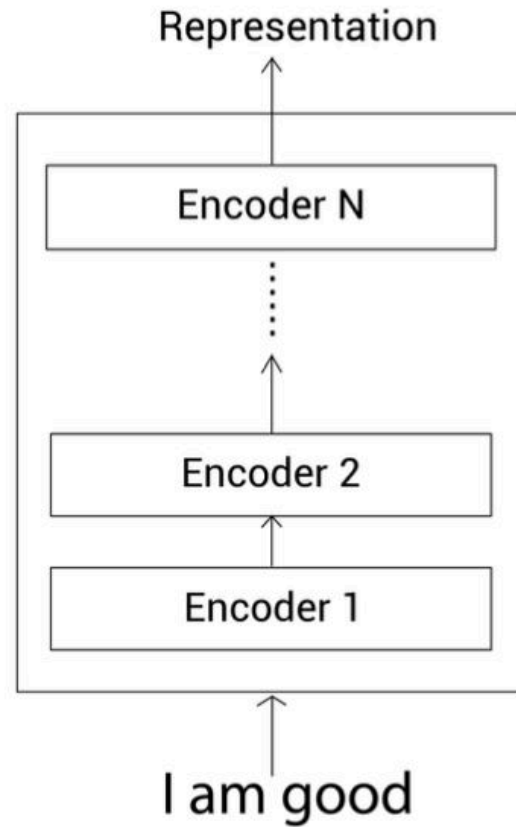


Figure 1.2 – A stack of N number of encoders



Architecture Overview

- **Seq2Seq Basis:** The model maps an input sequence to an output sequence without using recurrence or convolutions.
- **Encoder Stack:** Composed of multiple identical layers that generate continuous representations of the input sequence.
- **Decoder Stack:** Predicts the target sequence auto-regressively, heavily conditioned on the encoder's contextual output.
- **Core Modules:** Utilizes multi-head self-attention and position-wise feed-forward networks throughout the network.

Detailed Encoder Components

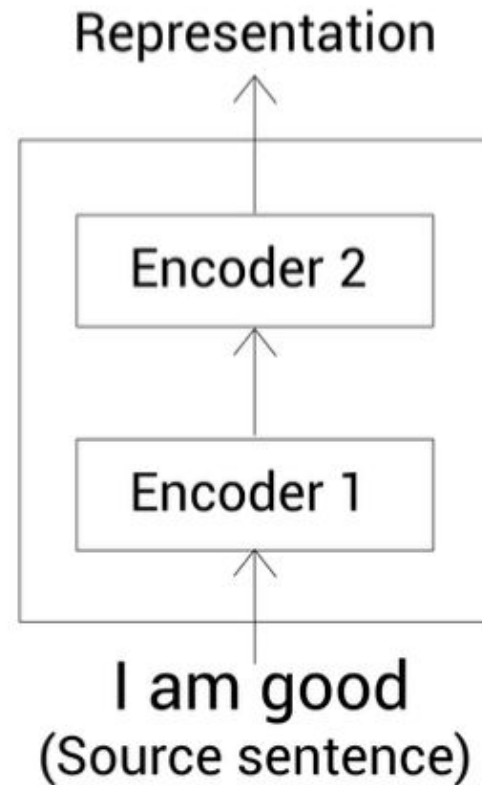


Figure 1.3 – A stack of encoders



Encoder Internals

- **Self-Attention Layer:** The first sub-layer allows the encoder to look at other words in the input sentence as it encodes a specific word, capturing context.
- **Feed-Forward Network:** The outputs of the self-attention layer are fed into a position-wise feed-forward neural network.
- **Independent Application:** The exact same feed-forward network is independently applied to each position in the sequence.
- **Vector Conversion:** The overall block processes the sequence of input words, ultimately converting them into a rich, continuous vector representation.

Complete Encoder-Decoder Architecture

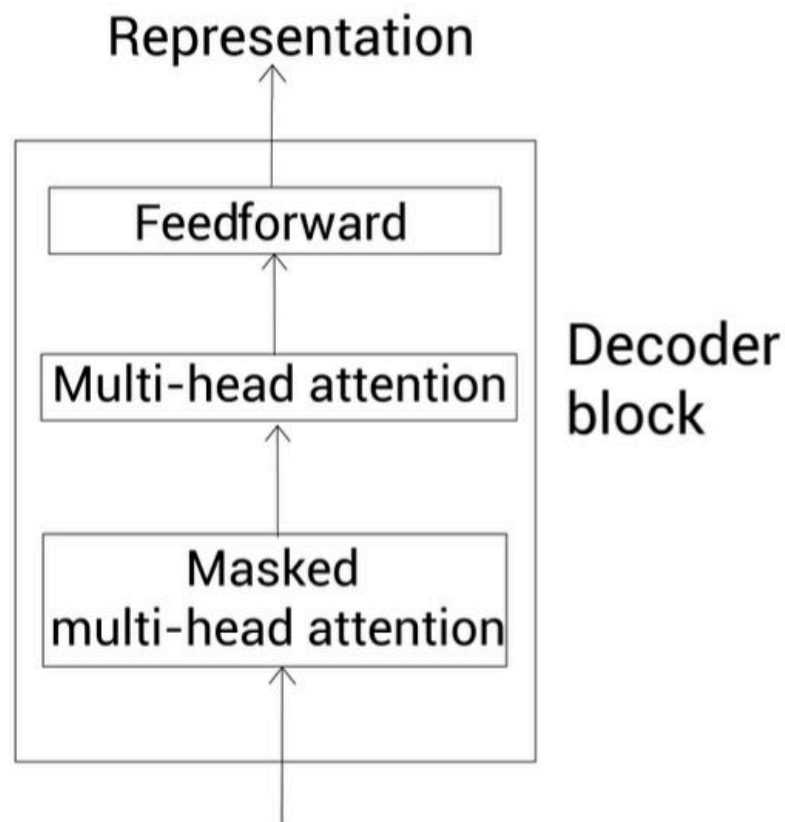


Figure 1.42 – A decoder block



Architecture Details

- **Encoder Stack:** Maps the input sequence into a continuous representation using Self-Attention and Feed-Forward networks.
- **Decoder Stack:** Generates the output sequence autoregressively, predicting one token at a time based on context.
- **Cross-Attention:** The decoder incorporates the encoder's final output, allowing it to focus on relevant input parts.
- **Residual & Norm:** Each sub-layer utilizes a residual connection followed by layer normalization for stable training.

Decoder Components Detailed View

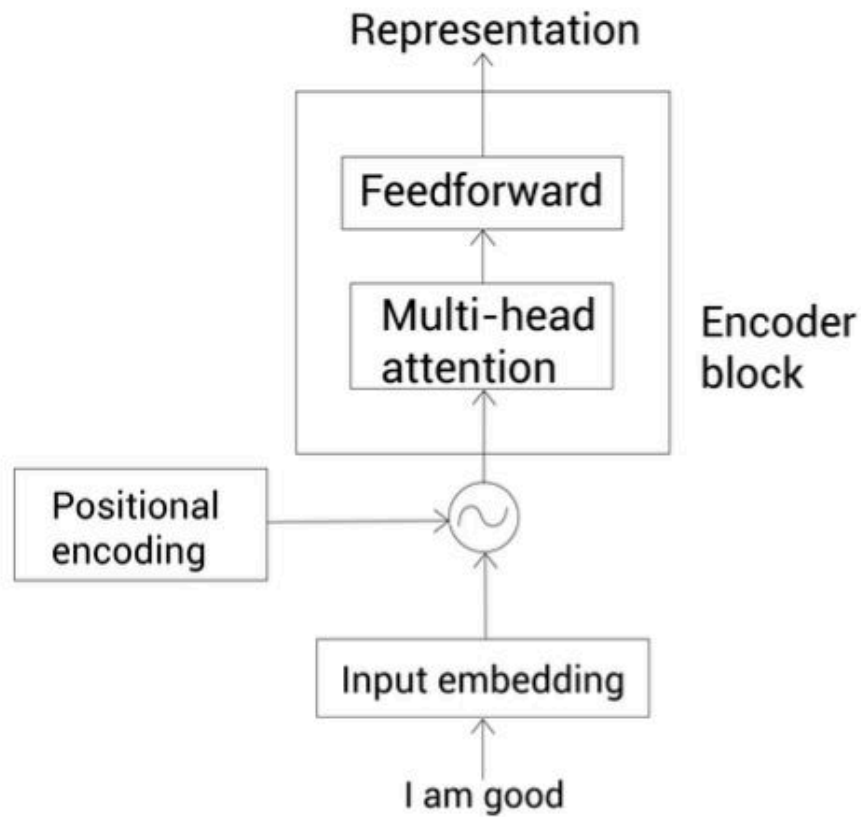


Figure 1.31 – A single encoder block

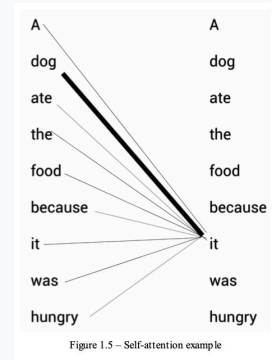


Decoder Modules

- **Masked Self-Attention:**
Ensures the model only attends to earlier positions in the sequence, preserving the autoregressive property for generation.
- **Cross-Attention Layer:**
Fuses information by using Queries from the decoder and Keys/Values from the encoder's output.
- **Feed-Forward Network:**
Applies a position-wise, fully connected layer independently to each position's representation.
- **Add & Norm:**
Residual connections combined with Layer Normalization wrap each sub-layer to stabilize deep training.

Attention Mechanism: Intuition and Types

Core Intuition: Learn where to focus in a sequence by weighting the importance of different parts dynamically.



Self-Attention

- Each token attends to **all other tokens** within the same sequence.
- Captures **internal relationships** and context (e.g., grammar, coreference).
- Queries (Q), Keys (K), and Values (V) all come from the **same source**.
- Can be **Masked (Causal)** for autoregressive tasks or **Unmasked (Bidirectional)**.

VS



Cross-Attention

- Tokens in one sequence attend to tokens in a **different sequence**.
- Found primarily in the **Decoder** to condition output on input.
- Queries (Q) come from the **Decoder**; Keys (K) and Values (V) come from the **Encoder**.
- Essential for tasks like **machine translation** (aligning source/target).

Scaled Dot-Product Attention

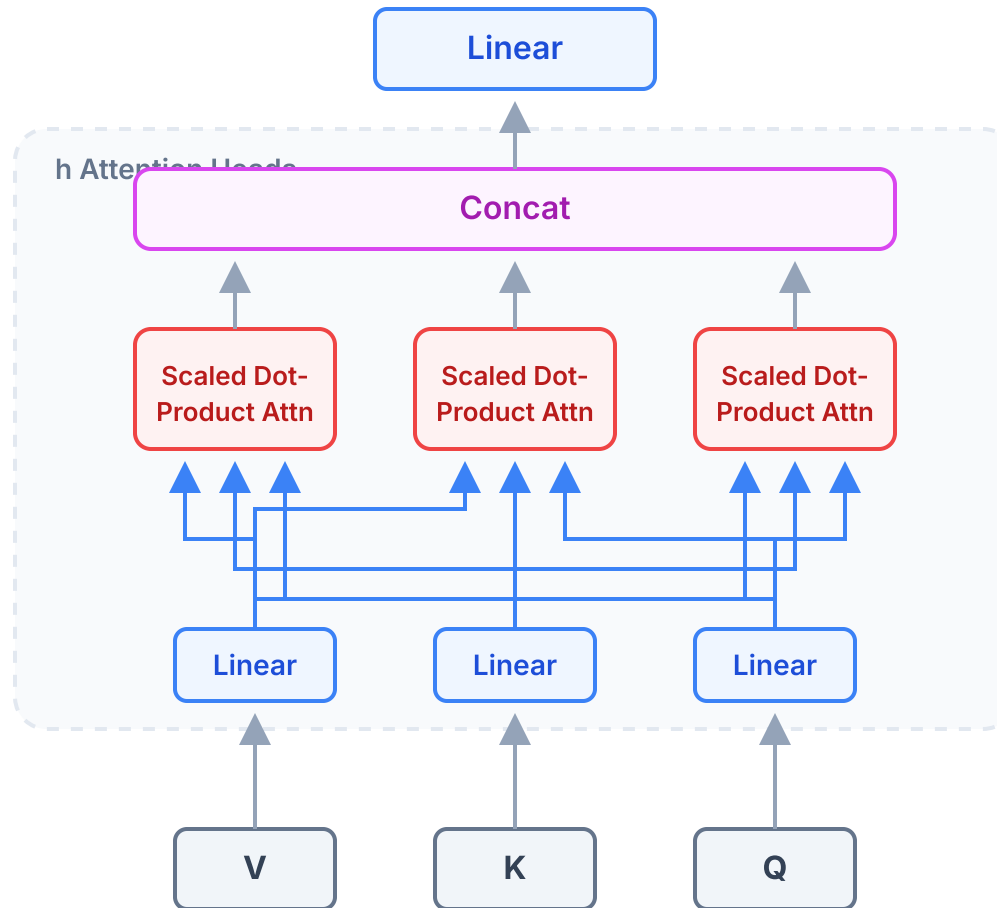
$\sqrt{x}$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Key Components

- **Q, K, V:** Queries, Keys, and Values matrices
- **d_k :** Dimension of the keys (and queries)
- **Scaling factor ($\sqrt{d_k}$):** Prevents the dot products from growing too large, which would push the softmax function into regions with extremely small gradients
- **Dot Product (QK^T):** Computes similarity scores between queries and keys
- **Softmax:** Normalizes scores into probabilities (weights) summing to 1
- **Optional Mask (M):** Can be added inside softmax to prevent attending to future tokens (causal masking)

Multi-Head Attention



Key Mechanics

- **Subspace Projections:** Multiple heads linearly project Q , K , and V into different lower-dimensional subspaces.
- **Parallel Computation:** Attention mechanisms run simultaneously across all projected heads.
- **Diverse Features:** Allows the model to jointly attend to information from different representation subspaces at different positions.
- **Aggregation:** Outputs are concatenated and passed through a final linear layer to restore dimensions.

Self-Attention Mechanism

CORE PRINCIPLES

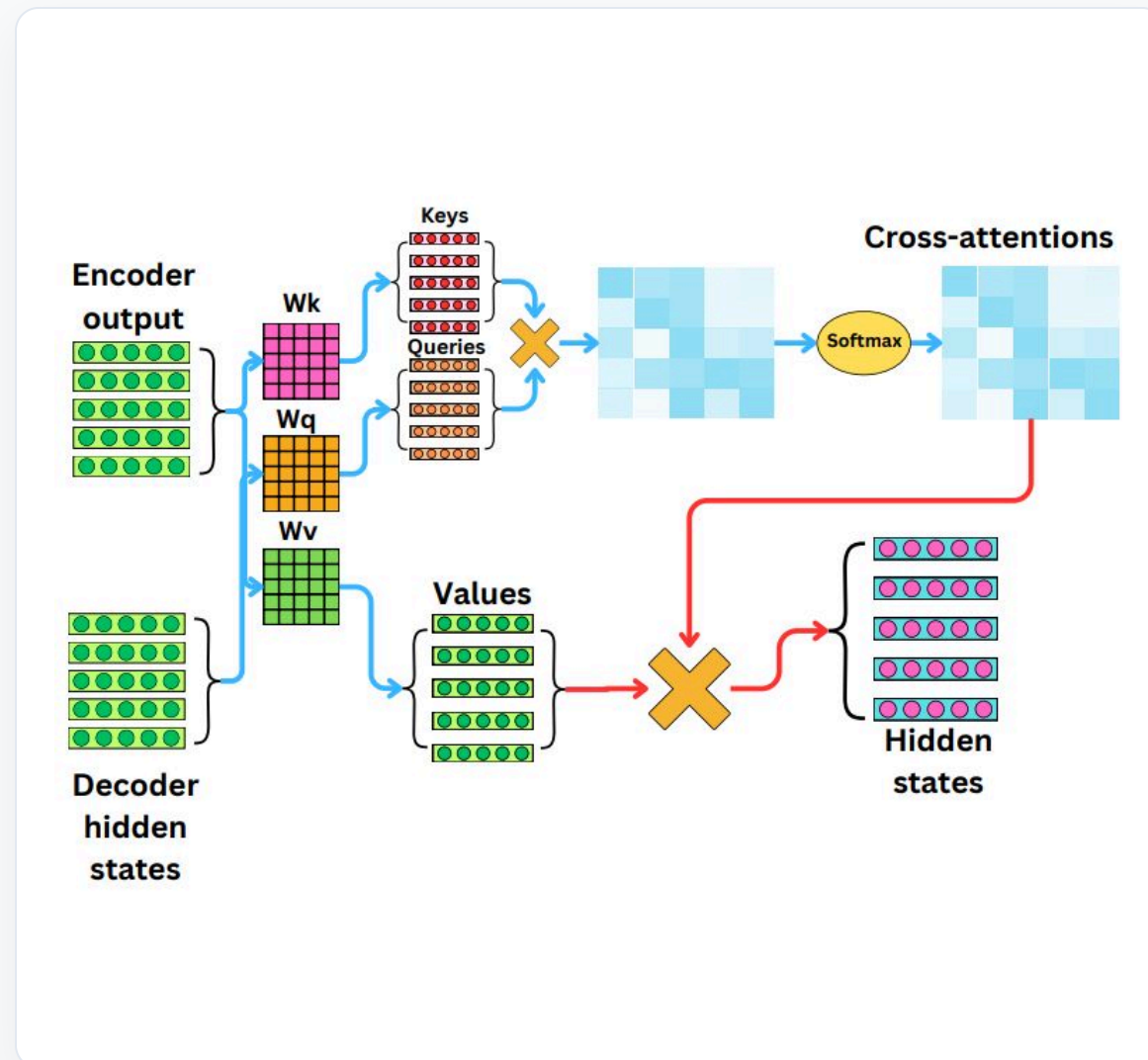
- **Global Context:** Each token in the sequence attends to all other tokens simultaneously to build a contextualized representation.
- **Long-Range Dependencies:** Overcomes limitations of RNNs by allowing direct connections between distant words in a sentence, regardless of their position.
- **Bidirectional Nature:** In architectures like BERT, the attention is unmasked, meaning the model looks at both past and future context to understand a word's meaning.
- **Dynamic Weights:** The attention scores determine how much focus (weight) one word should give to another when generating its new embedding.

 Self-Attention mechanism showing token-to-token attention

Cross-Attention Mechanism

ENCODER-DECODER ALIGNMENT

- **Information Bridge:** Cross-attention connects the two main stacks. It allows the decoder to focus on relevant parts of the encoded input sequence.
- **Routing Q, K, V:**
 - **Queries (Q):** Come from the previous decoder layer's output.
 - **Keys (K) & Values (V):** Come directly from the final output of the encoder stack.
- **Conditioned Generation:** By attending to the source sequence (via K and V), the model correctly conditions autoregressive generation on the input (critical for translation).
- **Unmasked:** Unlike the decoder's self-attention, cross-attention doesn't need to mask the encoder's



Masked vs Unmasked Attention

Attention Variants: How models control information flow to achieve different objectives, with an $O(n^2)$ complexity in sequence length.



Masked (Causal) Attention

- Prevents tokens from "looking ahead" at **future tokens**.
- Implemented by applying a **triangular mask** (setting upper triangle to $-\infty$) before softmax.
- Ensures valid **autoregressive generation** (predicting one word at a time).
- Primary mechanism in **GPT** and standard Transformer **Decoders**.

vs



Unmasked (Bidirectional)

- Allows full access to the **entire sequence context** simultaneously.
- No masking applied; tokens attend to both **left and right** context.
- Ideal for understanding sentence structure and **deep semantic meaning**.
- Primary mechanism in **BERT** and standard Transformer **Encoders**.

Attention Matrix Visualization

TOKEN RELATIONSHIPS

- **Mapping Dependencies:** The matrix visualizes the exact attention score between every pair of tokens in a sequence.
- **Weight Distribution:** Color intensity indicates the strength of the connection; darker/warmer areas show high attention focus.
- **Context Resolution:** Essential for understanding how the model resolves ambiguities and captures grammatical structure.
- **Self-Focus:** The strong diagonal line often indicates tokens attending heavily to themselves.

$$\begin{array}{l} \text{I} \\ \text{am} \\ \text{good} \end{array} \begin{bmatrix} 1.76 & 2.22 & \dots & 6.66 \\ 7.77 & 0.631 & \dots & 5.35 \\ 11.44 & 10.10 & \dots & 3.33 \end{bmatrix} \begin{array}{l} x_1 \\ x_2 \\ x_3 \end{array}$$

3x512

$\times$
input matrix
(embedding matrix)

Figure 1.6 – Input matrix

Attention Computation Details

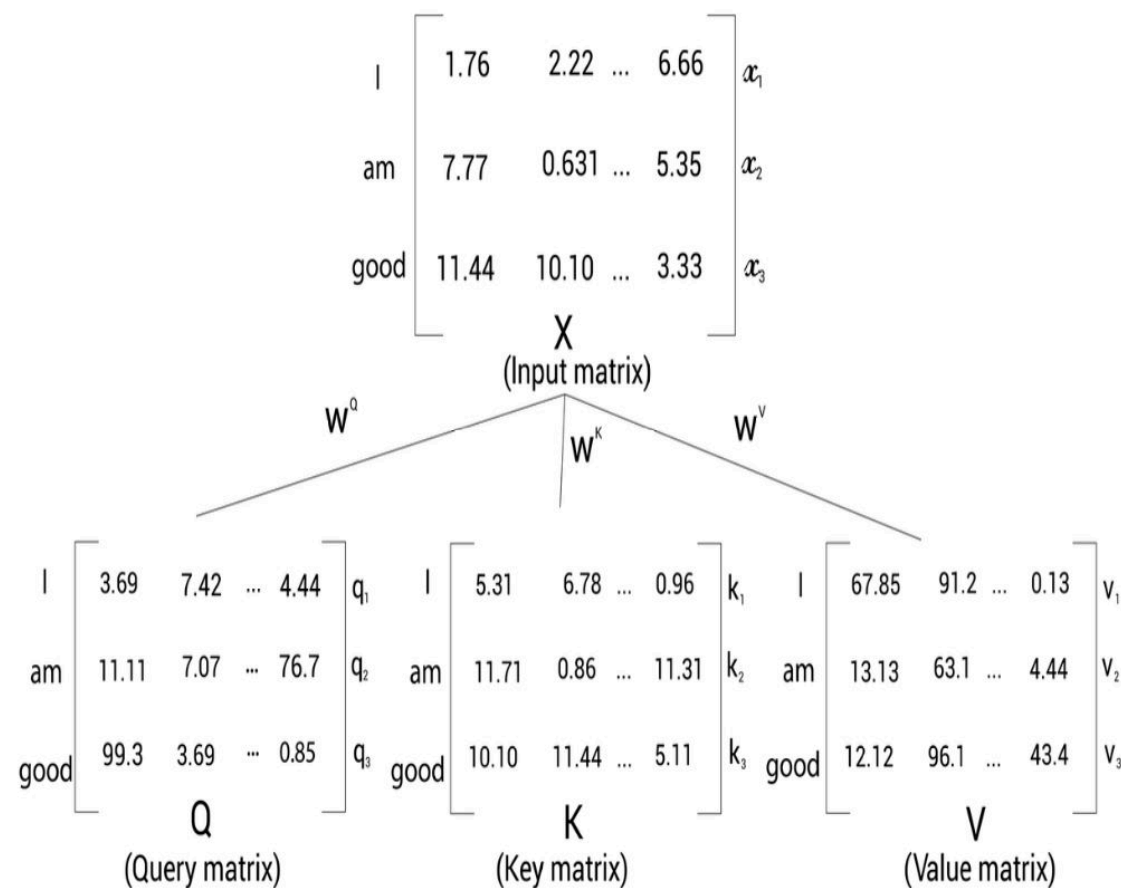


Figure 1.7 – Creating query, key, and value matrices



Computation Steps

- **1. Q, K, V Extraction:** Generate Query, Key, and Value vectors by multiplying input embeddings with learned weight matrices.
- **2. Calculate Scores:** Compute the dot product of the Query with all Keys to evaluate relevance across the sequence.
- **3. Scaling:** Divide scores by the square root of the dimension to maintain stable gradients during training.
- **4. Softmax:** Apply a softmax function to convert raw scores into positive normalized probabilities (summing to 1).
- **5. Weighted Sum:** Multiply each Value vector by its respective softmax score and sum them to produce the final output.

Multi-Head Attention: Detailed Breakdown

$$\begin{matrix} & \begin{matrix} I & am & good \end{matrix} \\ \begin{matrix} I \\ am \\ good \end{matrix} & \begin{bmatrix} 3.69 & 7.42 & \dots & 4.44 \\ 11.11 & 7.07 & \dots & 76.7 \\ 99.3 & 3.69 & \dots & 0.85 \end{bmatrix} \end{matrix} \cdot \begin{matrix} & \begin{matrix} I & am & good \end{matrix} \\ \begin{matrix} q_1 \\ q_2 \\ q_3 \end{matrix} & \begin{bmatrix} 5.31 & 11.71 & 10.10 \\ 6.78 & 0.86 & 11.44 \\ \vdots & \vdots & \vdots \\ 0.96 & 11.31 & 5.11 \end{bmatrix} \end{matrix} = \begin{matrix} & \begin{matrix} I & am & good \end{matrix} \\ \begin{matrix} I \\ am \\ good \end{matrix} & \begin{bmatrix} q_1.k_1 & q_1.k_2 & q_1.k_3 \\ q_2.k_1 & q_2.k_2 & q_2.k_3 \\ q_3.k_1 & q_3.k_2 & q_3.k_3 \end{bmatrix} \end{matrix}$$

Q

K^T

$QK^T = \begin{matrix} & \begin{matrix} I & am & good \end{matrix} \\ \begin{matrix} I \\ am \\ good \end{matrix} & \begin{bmatrix} 110 & 90 & 80 \\ 70 & 99 & 70 \\ 90 & 70 & 100 \end{bmatrix}$

Figure 1.10 – Computing the dot product between the query and key matrices

Process Flow

- **Multiple Projections:** The Q, K, and V vectors are linearly projected *h* times with differently learned weight matrices.
- **Independent Attention:** On each of these projected versions, attention is applied in parallel, yielding *h* output values.
- **Concatenation:** The outputs from each independent attention head are concatenated together into a single vector.
- **Final Linear Layer:** The concatenated output is passed through a final linear projection to match the expected model dimension.

Attention Example with Real Tokens

$$\frac{QK^T}{\sqrt{d_K}} = \frac{QK^T}{8} = \begin{array}{c} \text{I} \\ \text{am} \\ \text{good} \end{array} \begin{bmatrix} \text{I} & \text{am} & \text{good} \\ 13.75 & 11.25 & 10 \\ 8.75 & 12.375 & 8.75 \\ 11.25 & 8.75 & 12.5 \end{bmatrix}$$

Figure 1.14 – Dividing $Q \cdot K^T$ by the square root of d_K



Contextual Links

- **Contextual Awareness:** Each token computes attention scores against all other tokens in the sequence to build context.
- **Dynamic Weighting:** Thicker lines denote stronger connections. The strength varies based on the learned relevance between the words.
- **Resolving Ambiguity:** Essential for tasks like coreference resolution, enabling the model to link pronouns (like 'it') to specific entities correctly.
- **Visual Interpretation:** Attention heads can be visualized to debug and interpret exactly what linguistic features the model is focusing on.

Query, Key, and Value Mechanism

$$\text{Softmax}\left(\frac{QK^T}{\sqrt{d_K}}\right) = \begin{matrix} & \begin{matrix} \text{I} & \text{am} & \text{good} \end{matrix} \\ \begin{matrix} \text{I} \\ \text{am} \\ \text{good} \end{matrix} & \begin{bmatrix} 0.90 & 0.07 & 0.03 \\ 0.025 & 0.95 & 0.025 \\ 0.21 & 0.03 & 0.76 \end{bmatrix} \end{matrix}$$

Figure 1.15 – Applying the softmax function



Information Retrieval

- **Query (Q):** Represents the current token. It essentially asks, "What kind of information am I looking for?"
- **Key (K):** Represents all other tokens in the sequence. It acts as a label answering, "What information do I contain?"
- **Value (V):** The actual semantic content of each token that will be passed along if its Key matches the Query.
- **Matching:** The dot product of Q and K determines the attention score, dictating how much of V is retrieved.

Attention Visualization Patterns

$$Z = \begin{matrix} & \begin{matrix} I & am & good \end{matrix} \\ \begin{matrix} I \\ am \\ good \end{matrix} & \begin{bmatrix} 0.90 & 0.07 & 0.03 \\ 0.025 & 0.95 & 0.025 \\ 0.21 & 0.03 & 0.76 \end{bmatrix} \end{matrix}$$

$$V = \begin{matrix} & \begin{matrix} I & am & good \end{matrix} \\ \begin{matrix} I \\ am \\ good \end{matrix} & \begin{bmatrix} 67.85 & 91.2 & \dots & 0.13 \\ 13.13 & 63.1 & \dots & 4.44 \\ 12.12 & 96.1 & \dots & 43.4 \end{bmatrix} \end{matrix}$$

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

Figure 1.16 – Computing the attention matrix

Say we have the following:

$$Z = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} \begin{matrix} I \\ am \\ good \end{matrix}$$

Figure 1.17 – Result of the attention matrix



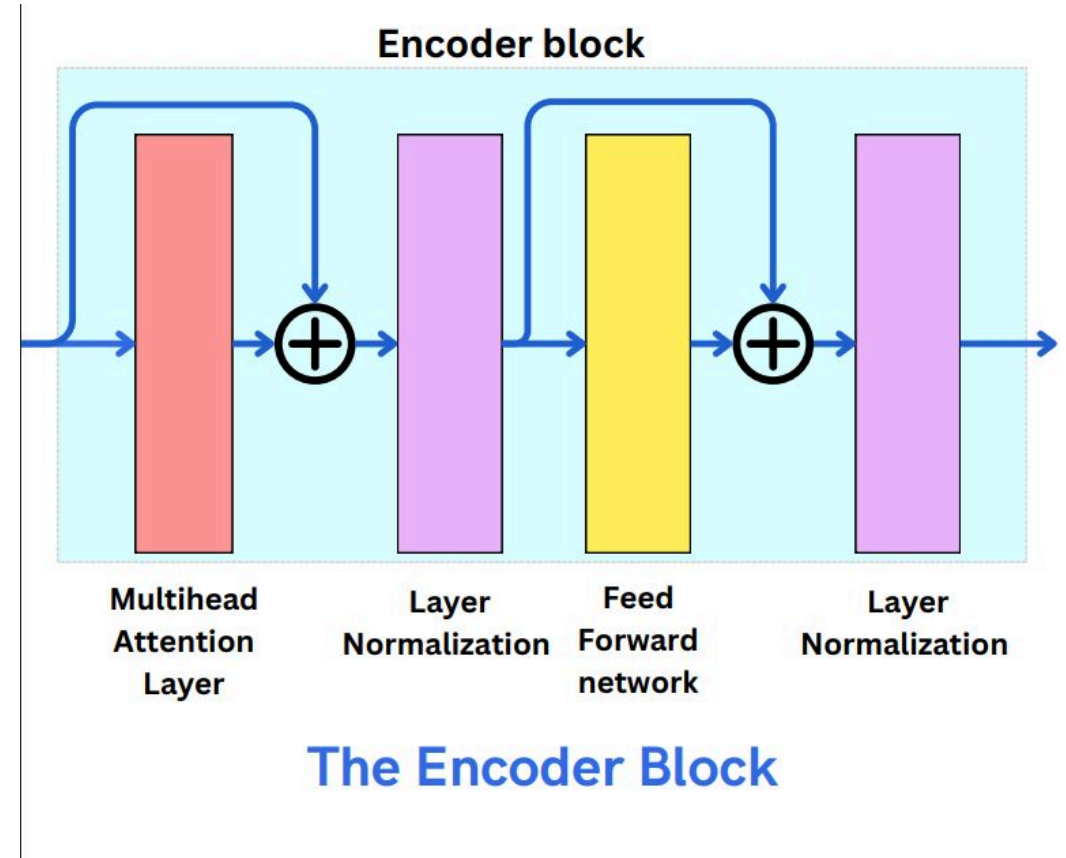
Pattern Insights

- Multi-Head Diversity:** Different attention heads learn to focus on distinct linguistic, syntactic, or structural relationships simultaneously.
- Local vs. Global Context:** Some heads naturally focus on adjacent words (local context) while others span across the entire sequence (global context).
- Syntactic Alignment:** Specific heads might track particular syntactic dependencies, such as pronoun-antecedent relationships or verb-object pairs.
- Interpretability:** Visualizing these connection patterns helps researchers interpret how transformers relate and process concepts over long distances.

Encoder Block: Structure

INTERNAL COMPONENTS

- **Layer Normalization (LN):** Applied before each sub-layer to stabilize the network and improve training speed.
- **Multi-Head Self-Attention:** Allows the model to jointly attend to information from different representation subspaces at different positions.
- **Residual Connections:** Skip paths around the sub-layers (Add & Norm) that help gradients flow smoothly and mitigate vanishing gradients.
- **Position-wise Feed-Forward Network (FFN):** Applies two linear transformations with a ReLU activation in between, identically to each position.
- The entire block is **repeated N times** (e.g., N=6 in the original paper) to form the complete encoder stack.



Layer Normalization in Transformers

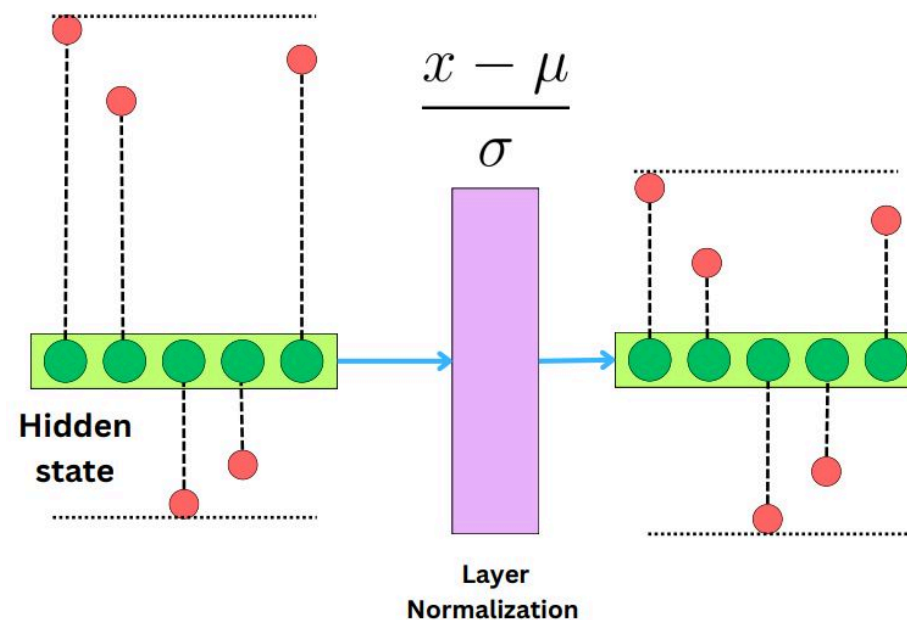
WHAT IS LAYER NORMALIZATION?

Layer Normalization is a critical mechanism that **stabilizes and accelerates training** in deep networks. In Transformers, it is applied **per token, across the hidden dimensions**, ensuring consistent input distributions to subsequent layers regardless of sequence length.



Key Characteristics

- **Per-Token Application:** Computes the mean and variance independently for each token representation in the sequence.
- **Pre-Norm Configuration:** Modern Transformers typically apply Layer Normalization *before* the Multi-Head Attention and Feed-Forward sublayers (Pre-Norm), rather than after, to improve gradient flow.
- **Learnable Parameters:** Introduces scale (gamma) and shift (beta) parameters that are learned during training to restore representational power if needed.



Position-wise Feed-Forward Network

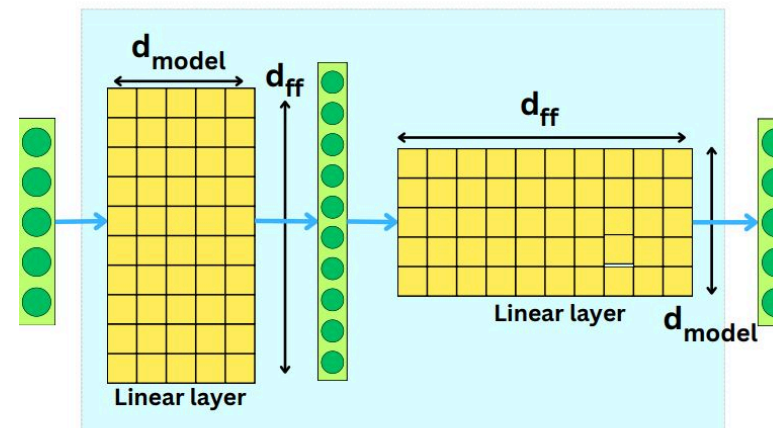
CORE COMPONENT DEFINITION

The **Position-wise Feed-Forward Network (FFN)** consists of two linear transformations with a non-linear activation function in between. It is applied **independently and identically** to every sequence position (token representation) after the attention sub-layer.



Key Characteristics and Functionality

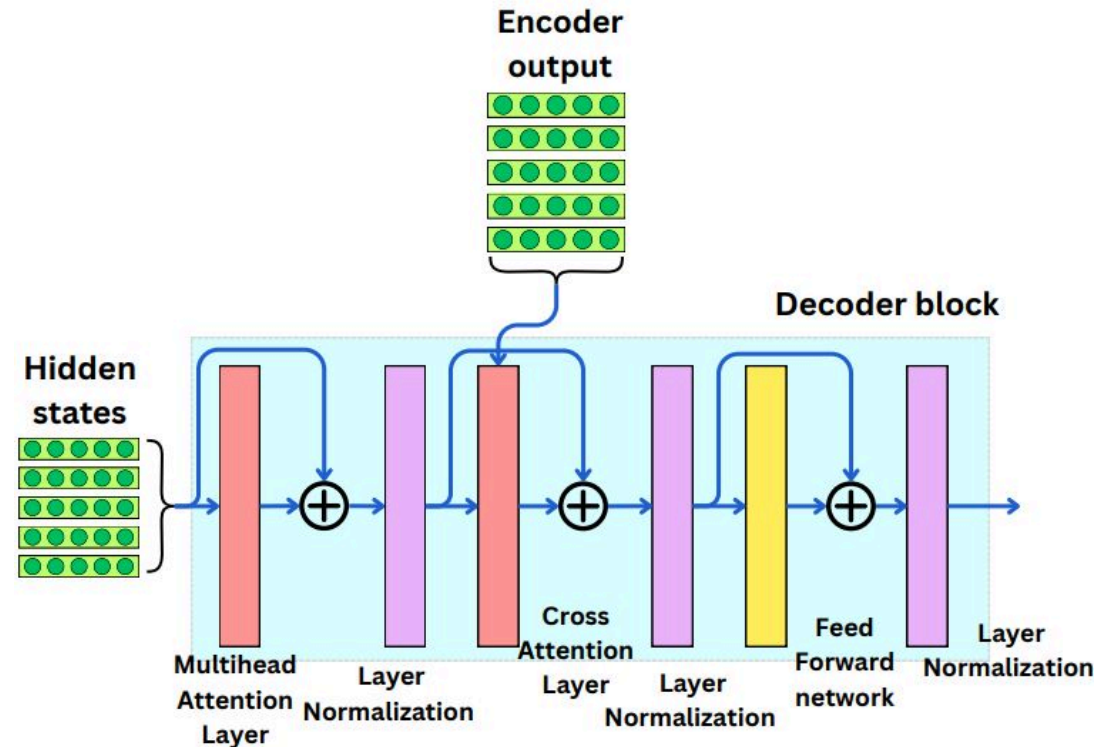
- **Two Linear Layers with Nonlinearity:** Typically uses ReLU or GELU activation between the layers (e.g., $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$).
- **Expands Hidden Dimensions:** The inner layer usually expands the hidden dimension significantly (often 4x the model dimension) to allow rich feature representations.
- **Applied at Each Position:** While the linear transformations are identical across all positions, they process each token's vector independently, acting as a non-linear feature extractor.



Decoder Block Structure

DECODER STACK

- **Layer Normalization (LN):** Applied before each sub-layer to stabilize and accelerate training.
- **Masked Self-Attention:** Prevents attending to future positions (causal masking).
 - Followed by Residual Connection
- **Cross-Attention (Encoder-Decoder):** Queries come from previous decoder layer, Keys and Values from encoder output.
 - Followed by Residual Connection
- **Position-wise Feed-Forward Network:** Two linear layers with nonlinearity.
 - Followed by Residual Connection
- Repeated **N times** to build the full decoder stack.



The Decoder Block

Masked Self-Attention in Decoder

AUTOREGRESSIVE GENERATION

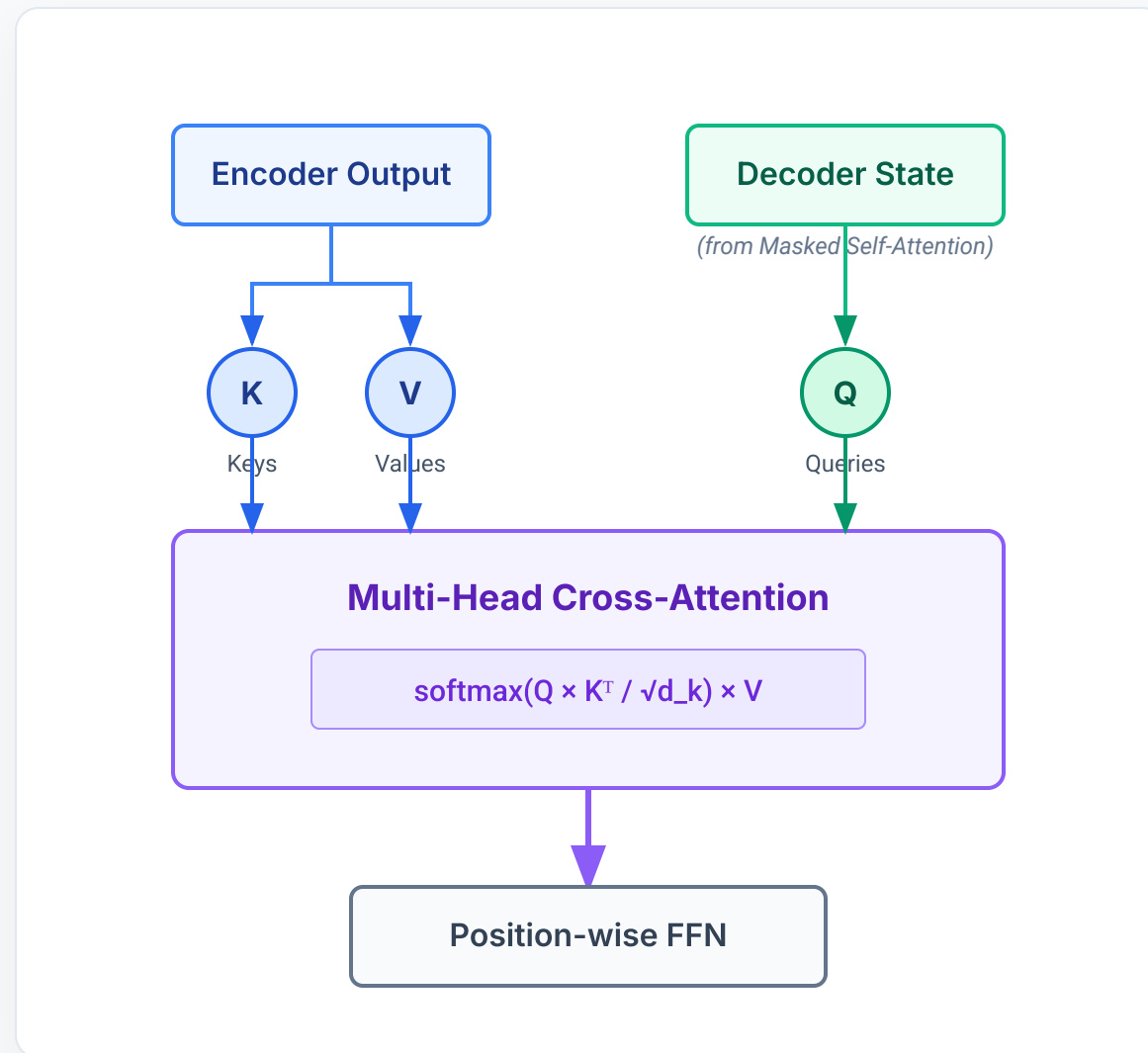
- **Autoregressive Property:** The decoder generates output sequence one token at a time, from left to right.
- **Preventing "Look-Ahead":** During training, we must prevent the model from "seeing" future words in the target sequence.
- **Triangle Mask:** Applied to the attention scores before softmax. It sets values corresponding to future tokens to $-\infty$.
- After softmax, these masked positions receive an attention weight of exactly **zero**.



Cross-Attention in Decoder

ENCODER-DECODER ALIGNMENT

- **Query Source:** Queries (**Q**) are generated from the previous decoder layer's internal states.
- **Key/Value Source:** Keys (**K**) and Values (**V**) come directly from the final output of the encoder stack.
- **Alignment Mechanism:** Dynamically aligns the target tokens currently being generated with relevant source content from the encoder.
- **Global Context:** Allows every position in the decoder to attend over *all* positions in the input sequence, unmasked.
- **Critical Role:** Essential for tasks like machine translation and summarization, where generation must be strictly conditioned on the source text.



Why Positional Encoding?

THE PROBLEM WITH SELF-ATTENTION

Unlike RNNs or CNNs, the self-attention mechanism is inherently **permutation-invariant**. It treats the input sequence as a "bag of words" and has no built-in notion of the order or position of tokens within the sequence.



Injecting Order Information

- **Addressing the Missing Sequence:** To make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens.
- **Positional Encodings:** "Positional encodings" are added to the input embeddings at the bottoms of the encoder and decoder stacks.
- **Dimension Matching:** The positional encodings have the same dimension (d_{model}) as the embeddings so they can be summed.
- **Long-Range Awareness:** Differentiates identical words based on position, capturing syntax and long-range dependencies.

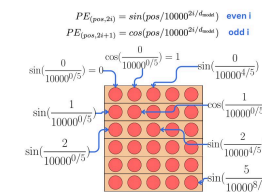
$$P = \begin{matrix} & \begin{bmatrix} \sin(0) & \cos(0) & \sin(0/100) & \cos(0/100) \\ \sin(1) & \cos(1) & \sin(1/100) & \cos(1/100) \\ \sin(2) & \cos(2) & \sin(2/100) & \cos(2/100) \end{bmatrix} \\ \text{am} & \\ \text{good} & \end{matrix}$$

Figure 1.29 - Computing the positional encoding matrix

Thus, our final positional encoding matrix, V , is given as follows:

$$P = \begin{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 \end{bmatrix} \\ \text{am} & \begin{bmatrix} 0.841 & 0.540 & 0.009 & 0.999 \end{bmatrix} \\ \text{good} & \begin{bmatrix} 0.909 & -0.416 & 0.019 & 0.999 \end{bmatrix} \end{matrix}$$

Figure 1.30 - Positional encoding matrix



Positional Encoding: Implementation



$$PE_{(pos, 2i)} = \sin(pos / 10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos / 10000^{2i/d_{model}})$$

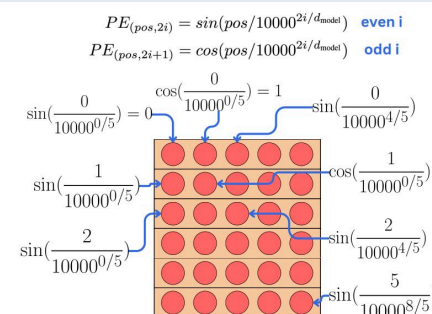
Key Components

- **pos**: Position of token in sequence (0, 1, 2, ...)
- **i**: Dimension index
- **d_{model}**: Dimensionality of embeddings
- **Sinusoidal Structure**: Uses sine for even dimensions and cosine for odd dimensions
- **Wavelengths**: Geometric progression from 2π to $10000 \cdot 2\pi$ across dimensions

Thus, our final positional encoding matrix, V , is given as follows:

$$P = \begin{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 \end{bmatrix} \\ \begin{matrix} am \\ good \end{matrix} & \begin{bmatrix} 0.841 & 0.540 & 0.009 & 0.999 \\ 0.909 & -0.416 & 0.019 & 0.999 \end{bmatrix} \end{matrix}$$

Figure 1.30 – Positional encoding matrix



Transformer Architecture: Detailed View

$$Z_1 = 0.90 \begin{bmatrix} 67.85 & 91.2 & \dots \end{bmatrix} + 0.07 \begin{bmatrix} 13.13 & 63.1 & \dots \end{bmatrix} + 0.03 \begin{bmatrix} 12.12 & 96.1 & \dots \end{bmatrix}$$

$v_1(I)$ $v_2(am)$ $v_3(good)$

Figure 1.18 – Self-attention of the word I

But how is this useful? To answer this question, let's take a little detour to the example sentence we saw earlier, *A dog ate the food because it was hungry*. Here, the word *it* indicates *dog*. To compute the self-attention of the word *it*, we follow the same preceding steps. Suppose we have the following:

$$Z_{it} = 0.0 \begin{bmatrix} 71.1 & 6.1 & \dots \end{bmatrix} + 1.0 \begin{bmatrix} 31.1 & 11.1 & \dots \end{bmatrix} + \dots + 0.0 \begin{bmatrix} 0.9 & 11.44 & \dots \end{bmatrix} + \dots + 0.0 \begin{bmatrix} 0.8 & 12.44 & \dots \end{bmatrix}$$

$V_1(A)$ $V_2(dog)$ $V_5(food)$ $V_9(hungry)$

Figure 1.19 – Self-attention of the word it



Internal Components

- **Self-Attention Blocks:** Mechanism allowing the model to jointly attend to information from different representation subspaces at different positions.
- **Masked Attention:** The decoder utilizes masking to prevent future tokens from being accessed during autoregressive generation, preserving order.
- **Add & Norm Layers:** Residual connections followed by layer normalization stabilize training and prevent vanishing gradients in deep networks.
- **Feed-Forward Networks:** Position-wise transformations applied independently to each token, introducing vital non-linearities via ReLU or GELU activations.

Transformer Detail View

1. First, we compute the dot product between the query matrix and the key matrix, QK^T , and get the similarity scores.
2. Next, we divide QK^T by the square root of the dimension of the key vector, $\sqrt{d_k}$.
3. Then, we apply the softmax function to normalize the scores and obtain the score matrix, $\text{softmax}(QK^T / \sqrt{d_k})$.
4. At the end, we compute the attention matrix, Z , by multiplying the score matrix by the value matrix, V .



Deep Dive into Layers

- **Sub-layers:** Each block contains Multi-Head Attention and Position-wise Feed-Forward Networks.
- **Residual Connections:** 'Add & Norm' operations follow each sub-layer, stabilizing gradients during deep training.
- **Layer Normalization:** Applied over the hidden dimension, independently for each token to improve convergence.
- **Information Flow:** Outputs from one layer seamlessly feed into the inputs of the next layer within the stack.

Transformer Architecture Overview

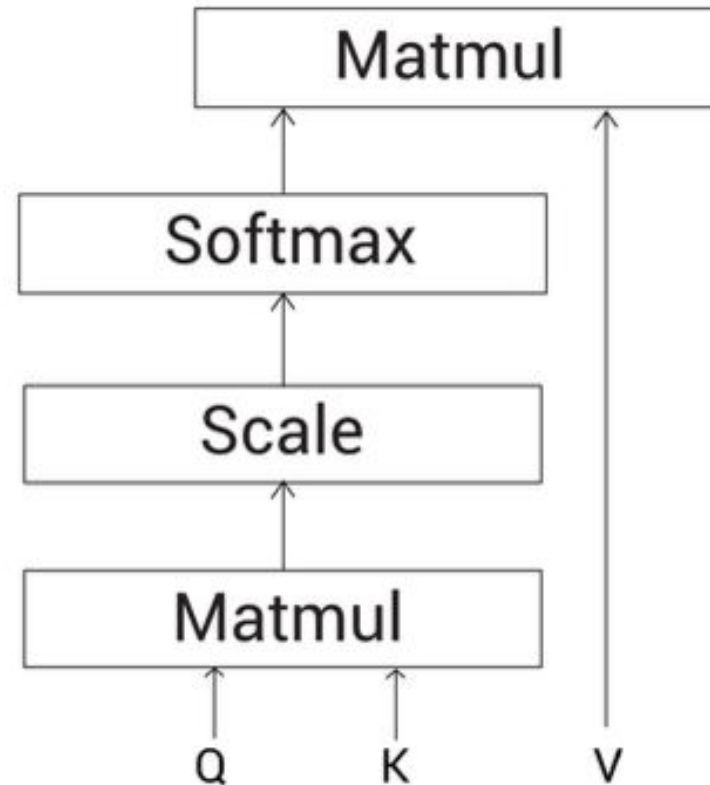


Figure 1.22 – Self-attention mechanism



High-Level Flow

- **Input Processing:** Source sequence is embedded and positionally encoded before entering the encoder block.
- **Encoder Outputs:** The encoder stack processes the inputs to create a set of continuous representations, providing rich context mapping.
- **Decoder Integration:** The decoder uses its own autoregressive inputs and actively attends to the encoder's final representations.
- **Output Generation:** A final linear layer and softmax generate discrete target token probabilities one step at a time.

Transformer Architecture Details

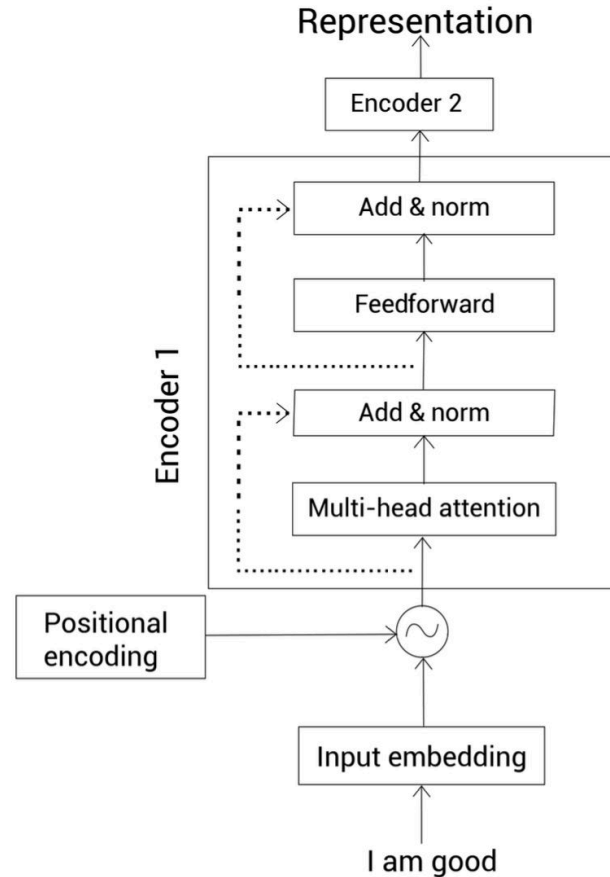
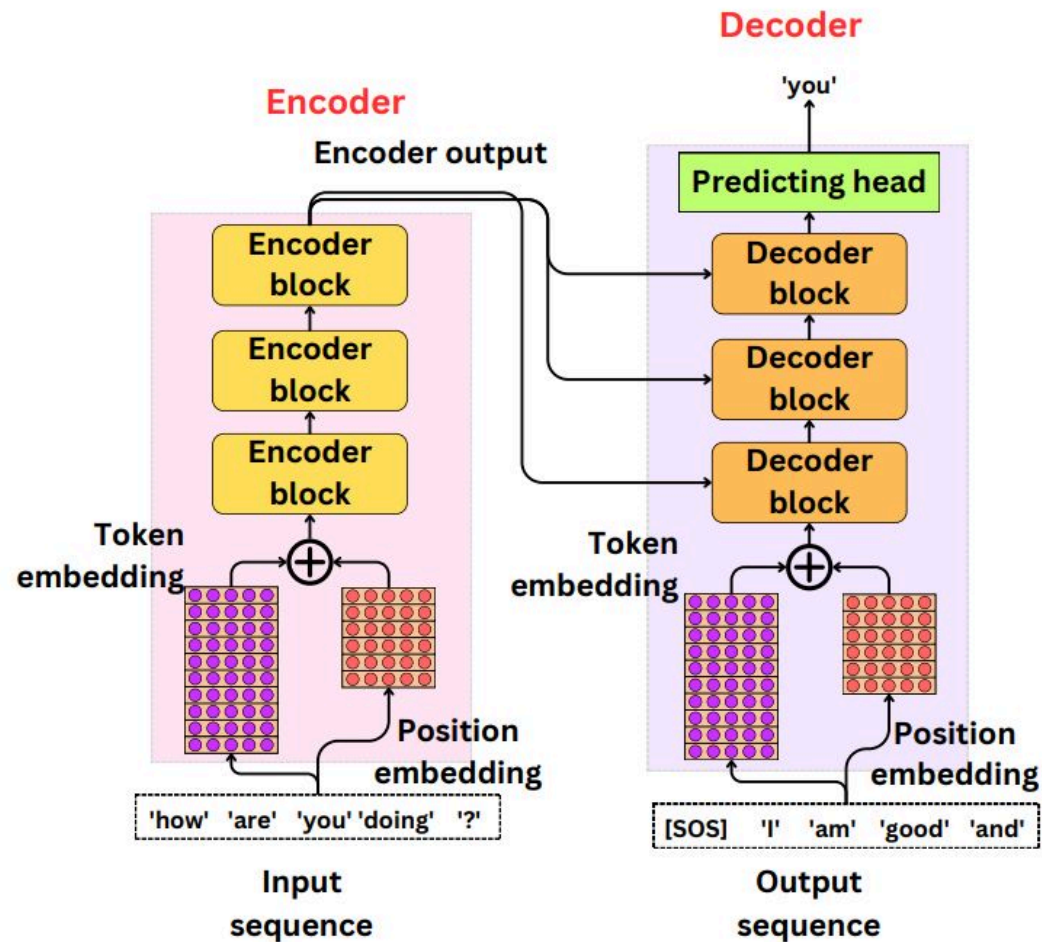


Figure 1.34 – A stack of encoders with encoder 1 expanded

Key Mechanisms

- **Sub-layer Integration:** Each block systematically contains sub-layers dedicated to attention and feed-forward processing, structured for efficient learning.
- **Residual Connections:** Bypasses the sub-layers (Add & Norm) to prevent vanishing gradients and stabilize deep network training.
- **Modular Stacking:** The architecture repeats identical layers (N times) to extract hierarchical, complex features from sequences.
- **Information Flow:** Ensures stable gradient propagation and maintains rich contextual representations across the model depth.

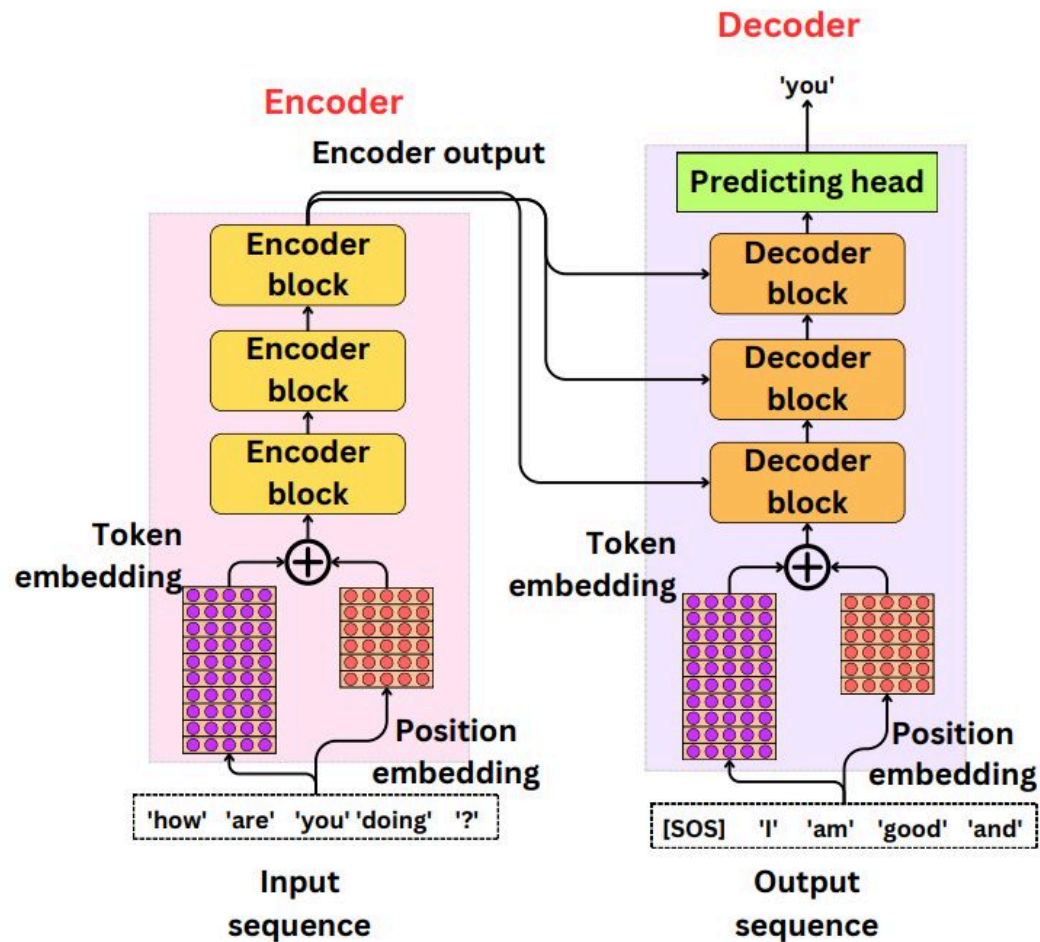
Full Transformer Architecture



End-to-End Structure

- **Encoder Stack:** Processes input tokens into continuous representations, mapping source sequences into hidden dimensions.
- **Decoder Stack:** Auto-regressively generates output tokens, attending to both previous outputs and encoder context.
- **Information Flow:** Skip connections and layer normalization stabilize deep learning layers throughout the blocks.
- **Predicting Head:** Final linear layer and softmax generate probability distributions over the target vocabulary.

Transformer Architecture Details



Component Deep Dive

- **Sub-layer Integration:** Each sub-layer in the encoder and decoder features a residual connection around it.
- **Layer Normalization:** Applied subsequently to each residual connection to stabilize the hidden state dynamics.
- **Dimensionality:** To facilitate the residual connections, all sub-layers and embedding layers output data of dimension $\backslash (d_{\text{model}} = 512 \backslash)$.
- **Information Flow:** The architecture efficiently routes information through linear projections and parallel multi-head attention blocks.

Transformer Architecture: Predicting Head

 Transformer Predicting Head Architecture

Output Generation

- **Linear Projection:** The final decoder output vector is passed through a fully connected linear layer.
- **Vocabulary Expansion:** The linear layer projects the hidden representation into a larger logits vector, matching the size of the model's entire vocabulary.
- **Softmax Layer:** Converts the raw, unnormalized logits into valid probabilities (all positive values that sum to exactly 1.0).
- **Token Selection:** The word associated with the highest probability is selected (greedy decoding) or sampled as the final output token for this time step.

BERT: Transformer Encoder



Source: Jay Alamar (Illustrated BERT)



What is BERT?

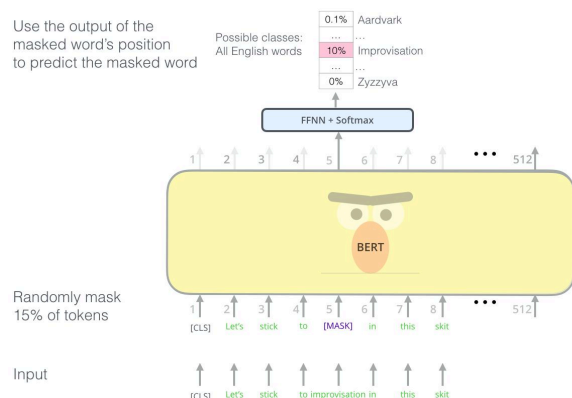
- **Encoder Stack:** BERT is fundamentally a trained stack of Transformer Encoder blocks.
- **Bidirectional Context:** Unlike traditional models, it processes all tokens in the sequence at once, looking both forward and backward.
- **Contextual Embeddings:** The final output is a sequence of rich, context-aware vector representations for each word.
- **Foundation:** Serves as a pre-trained base that can be fine-tuned for a wide variety of downstream NLP tasks.

BERT Pre-Training: MLM and NSP

Pre-training Strategies: BERT learns deep bidirectional representations using two concurrent self-supervised tasks on large unannotated text corpora.

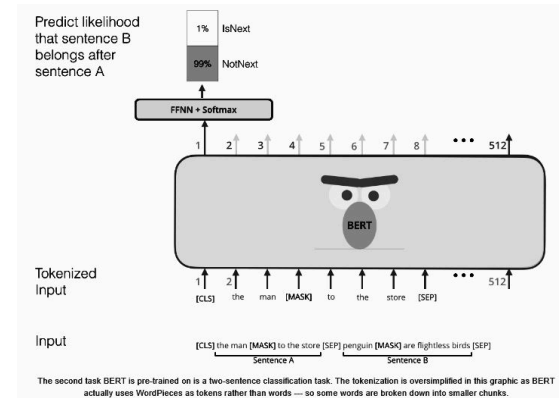
Masked Language Modeling

- Masks **15% of tokens** in the input sequence at random.
- Goal is to predict the missing word based on the bidirectional context.
- **The 80/10/10% Rule:** For the chosen 15%:
 - **80%:** Replace with `[MASK]` token
 - **10%:** Replace with a random word
 - **10%:** Keep the original word



Next Sentence Prediction

- Binary classification task to understand **sentence relationships**.
- Given sentences A and B, is B the actual next sentence that follows A?
- **Training Distribution:**
 - **50%:** B is the actual next sentence
 - **50%:** B is a random sentence from corpus
- Crucial for downstream tasks like **Question Answering** and NLI.



AND

BERT Fine-Tuning and Embeddings



APPROACH 1

Task Fine-Tuning

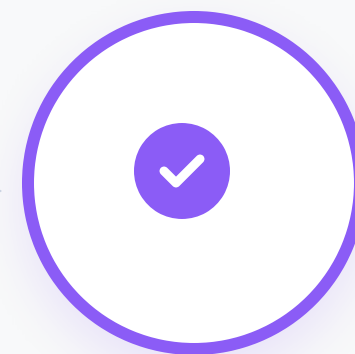
Fine-tune the entire pre-trained model for specific tasks (e.g., QA, NER). No masking applied during this phase. Attach a task-specific head.



APPROACH 2

Contextual Embeddings

Use the pre-trained BERT purely as an embedding layer to generate rich, context-aware representations for downstream models.



APPLICATIONS

Downstream Tasks

Apply the customized model to typical NLP tasks: classification, natural language inference, question answering, etc.

DistilBERT: Knowledge Distillation

LIGHTER, FASTER BERT

- Developed and open-sourced by the team at **HuggingFace**.
- A smaller, faster, and lighter version of BERT that roughly matches its performance.
- Based on the concept of **knowledge distillation**, where a larger "teacher" model trains a smaller "student" model.
- Trained only using **MLM (Masked Language Model)**, dropping the NSP objective.
- **Example Use Case:** Sentence Classification. We use pre-trained DistilBERT and only train a logistic regression head on top.



BERT Configurations and Applications



BERT-Base

- **Layers (L):** 12 Transformer blocks
- **Hidden Size (H):** 768 dimensions
- **Attention Heads (A):** 12 multi-head attention blocks
- **Total Parameters:** 110 Million

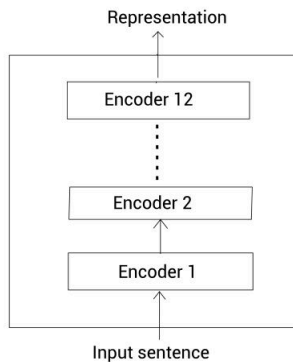


Figure 2.5 – BERT-base



BERT-Large

- **Layers (L):** 24 Transformer blocks
- **Hidden Size (H):** 1024 dimensions
- **Attention Heads (A):** 16 multi-head attention blocks
- **Total Parameters:** 340 Million

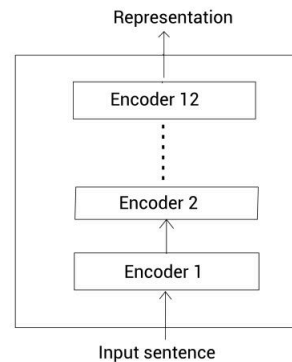


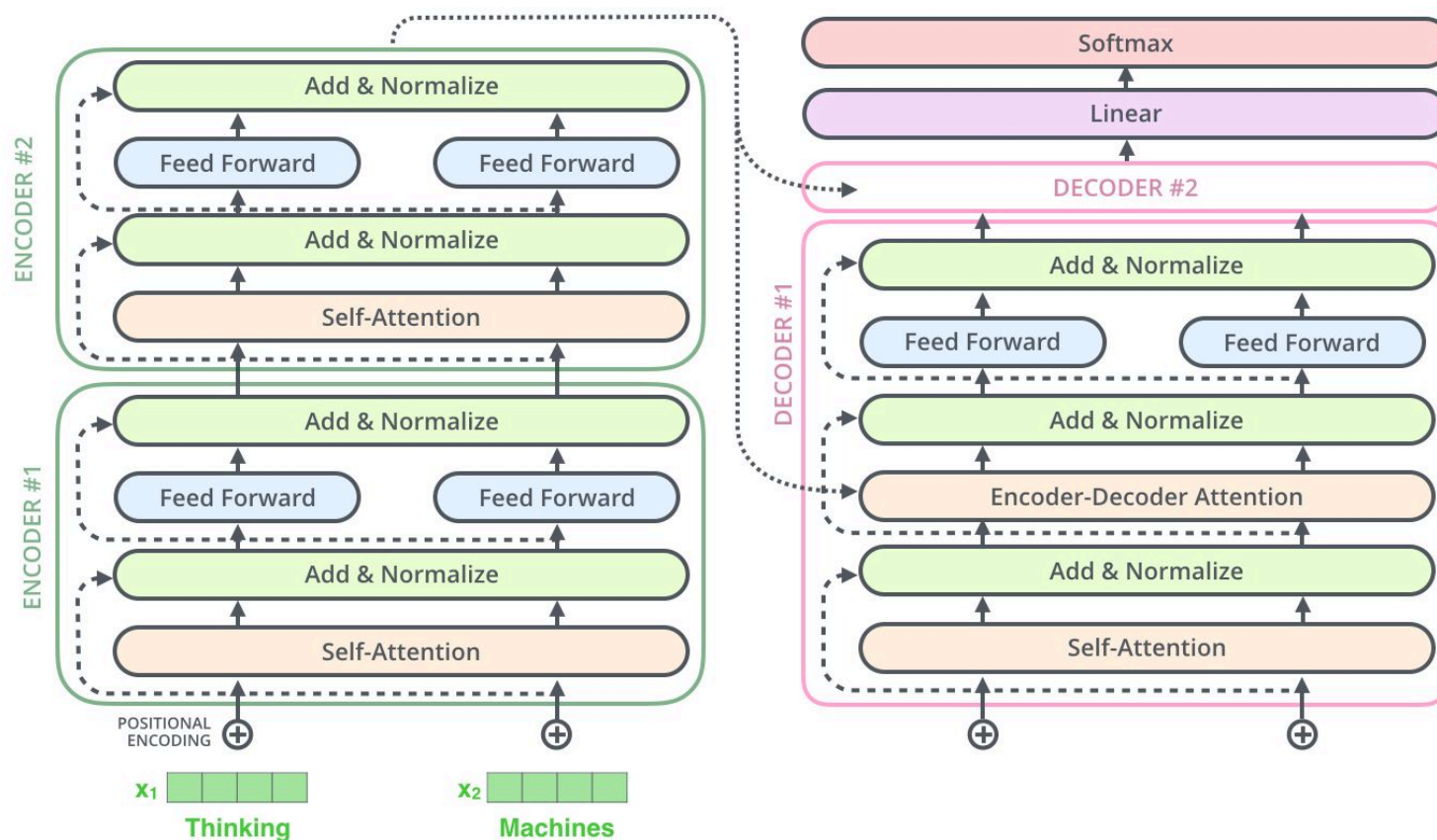
Figure 2.5 – BERT-base



Applications

- **Sentence Classification:** Sentiment analysis, intent detection, etc.
- **Token Classification:** Named Entity Recognition (NER), Part-of-Speech tagging.
- **Question Answering:** Extracting answers from a context paragraph.
- **Textual Entailment:** Determining relationship between two sentences.

BERT Internal Architecture



Bidirectional Encoder

- **Encoder Stack:** BERT is fundamentally composed of a trained stack of Transformer encoders.
- **Bidirectional Context:** Unlike traditional models, BERT looks at and processes all tokens at once to understand full context.
- **Self-Attention:** Heavily relies on multi-head self-attention to relate each word to every other word in the sequence.
- **Residual Pathways:** Maintains residue connections and layer normalization inside its encoder blocks for stable training.

BERT Attention Patterns

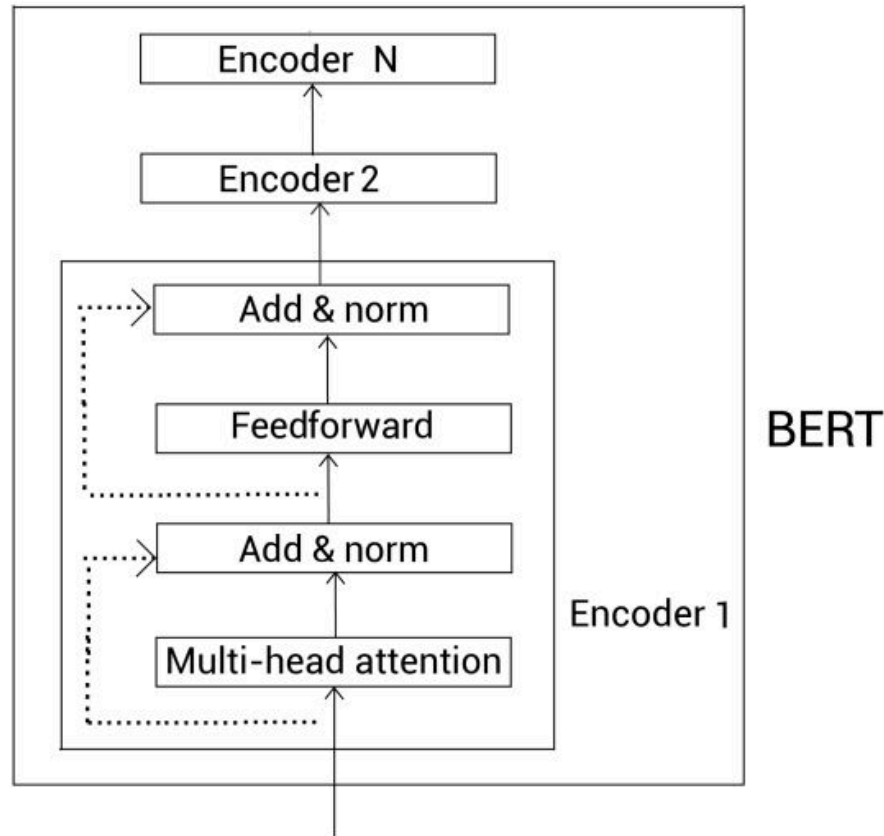


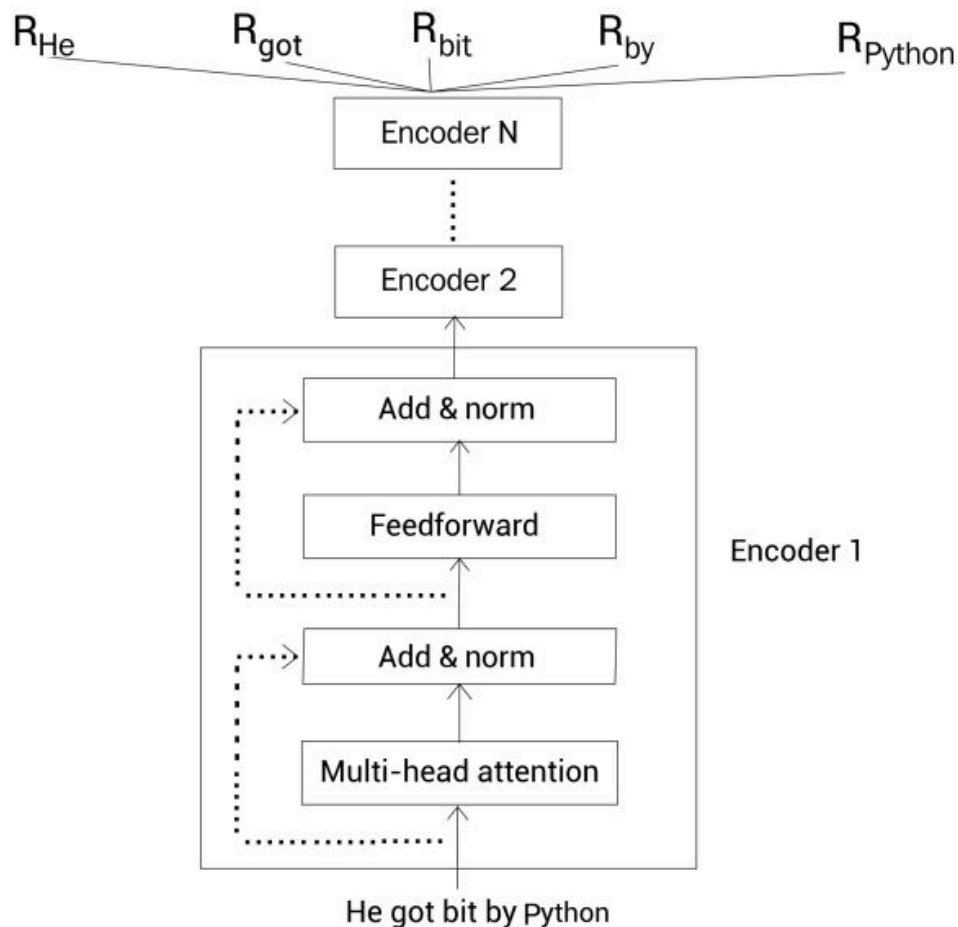
Figure 4.1 – BERT with N number of encoders



Multi-Head Self-Attention

- **Bidirectional Context:** BERT is bidirectional because it processes and produces all tokens at once, rather than sequentially.
- **Multi-Head Mechanism:** Multiple attention heads allow the model to jointly attend to information from different representation subspaces.
- **Dynamic Focus:** Each token dynamically determines which other tokens in the sequence to focus on, resolving ambiguity.
- **Rich Dependencies:** This parallel attention process effectively captures complex contextual dependencies and relationships across the entire sequence.

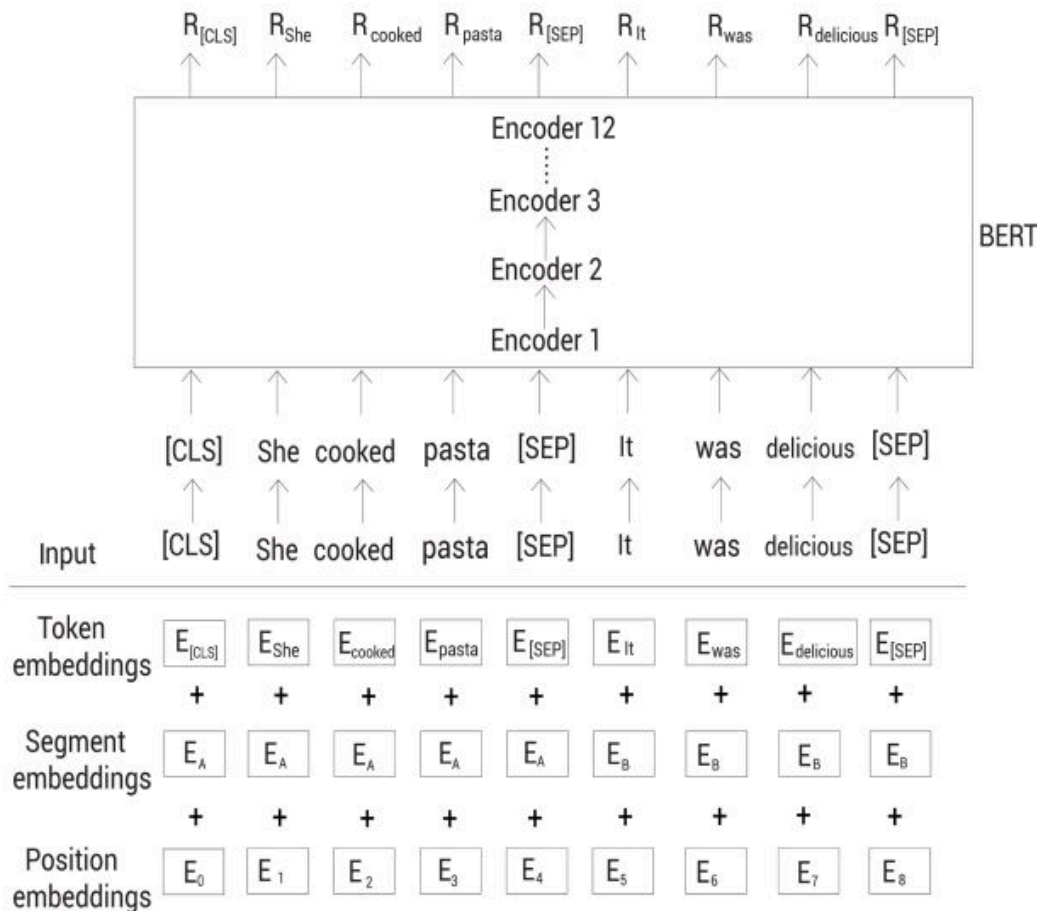
BERT Word Representation Examples



Contextual Embeddings

- **Contextual Meaning:** BERT generates a unique representation for each word based dynamically on its surrounding context in the sentence.
- **Self-Attention Mechanism:** The model relates the target word to all other words in the sequence to compute its final embedding.
- **Bidirectional Analysis:** It simultaneously looks at both the left and right context, avoiding the limitations of unidirectional left-to-right models.
- **Word Disambiguation:** Because embeddings are dynamic, the exact same word receives completely different vector mappings depending on its usage and meaning in different

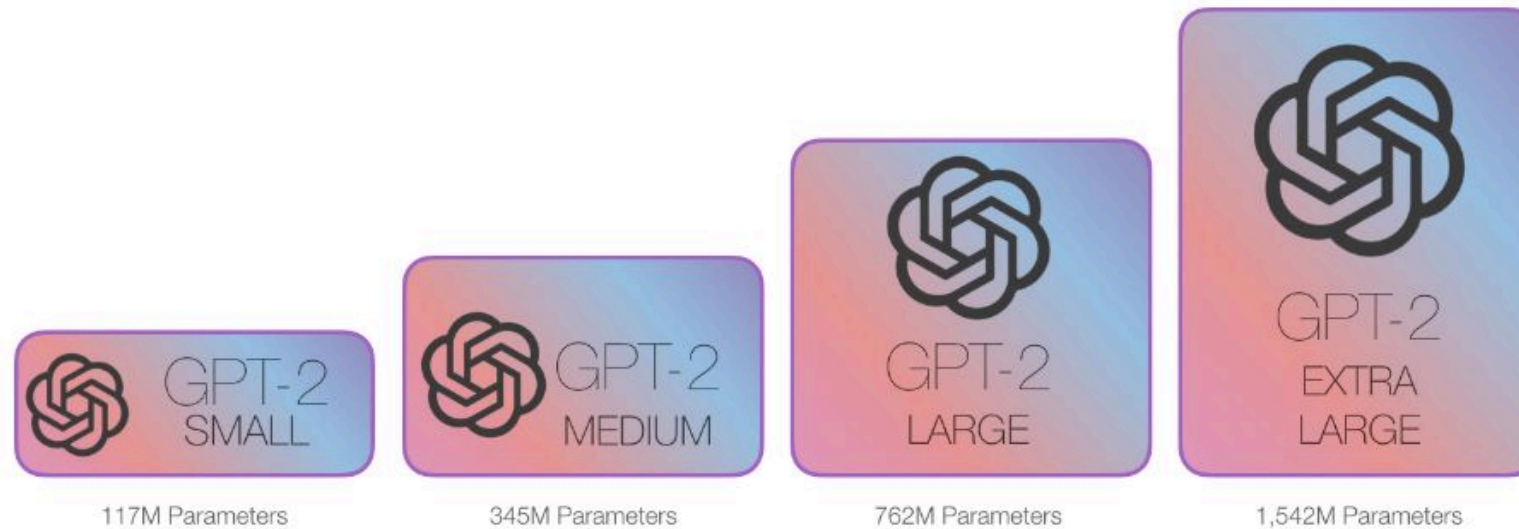
BERT Full Architecture View



Model Structure

- **Transformer Encoder:** BERT is fundamentally composed of a multi-layer bidirectional Transformer encoder stack.
- **Bidirectional Context:** Unlike traditional models, it reads the entire sequence simultaneously, capturing deep context from both sides.
- **Deep Architecture:** Utilizes extensive layers of self-attention blocks to process representations layer by layer.
- **Task Agnostic:** The underlying architecture remains identical during pre-training, enabling easy fine-tuning for diverse downstream tasks.

GPT: Transformer Decoder



Generative Model

- **Autoregressive:** Generates text by predicting the next token in a sequence based on previous tokens.
- **Decoder-Only:** Unlike the original Transformer, GPT drops the cross-attention layers and uses only the decoder stack.
- **Causal Masking:** Crucially employs masked self-attention to ensure predictions only depend on past, not future, tokens.
- **One Token at a Time:** The output is generated iteratively, appending the predicted token back into the input for the next step.

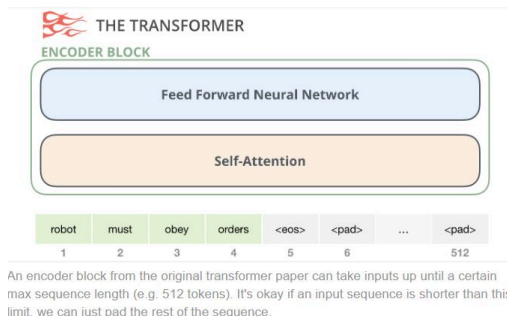
GPT Architecture Differences

Key Distinction: GPT simplifies the traditional Transformer by using a decoder-only architecture optimized for autoregressive language modeling.



Traditional Transformer

- Consists of both **Encoder** and **Decoder** stacks.
- Decoder relies on **cross-attention** to process encoder outputs.
- Encoder uses **unmasked (bidirectional)** self-attention.
- Primarily designed for **sequence-to-sequence** tasks.

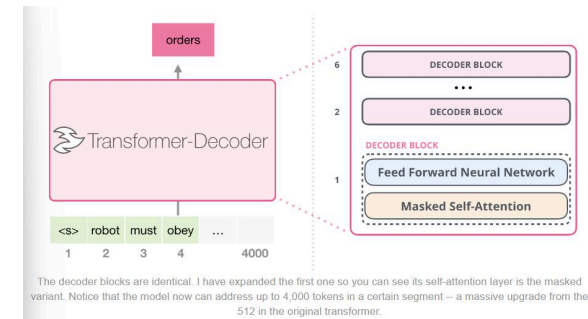


vs



GPT (Decoder-Only)

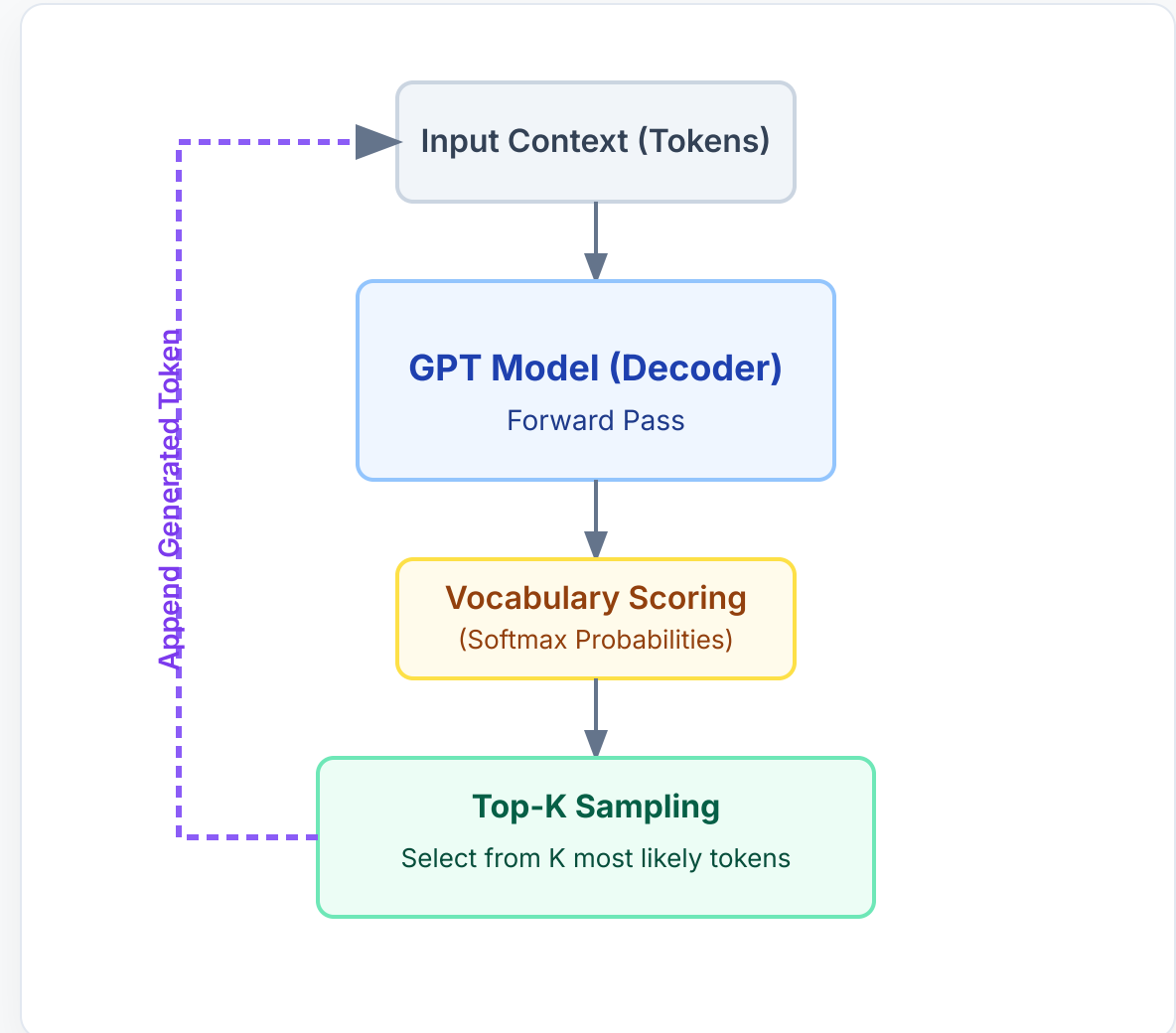
- Uses a **Decoder-only** stack with a language modeling head.
- **Cross-attention is dropped** entirely as there is no encoder block.
- Relies exclusively on **masked self-attention**.
- Optimized for autoregressive **next-token prediction**.



Generation and Sampling Strategies

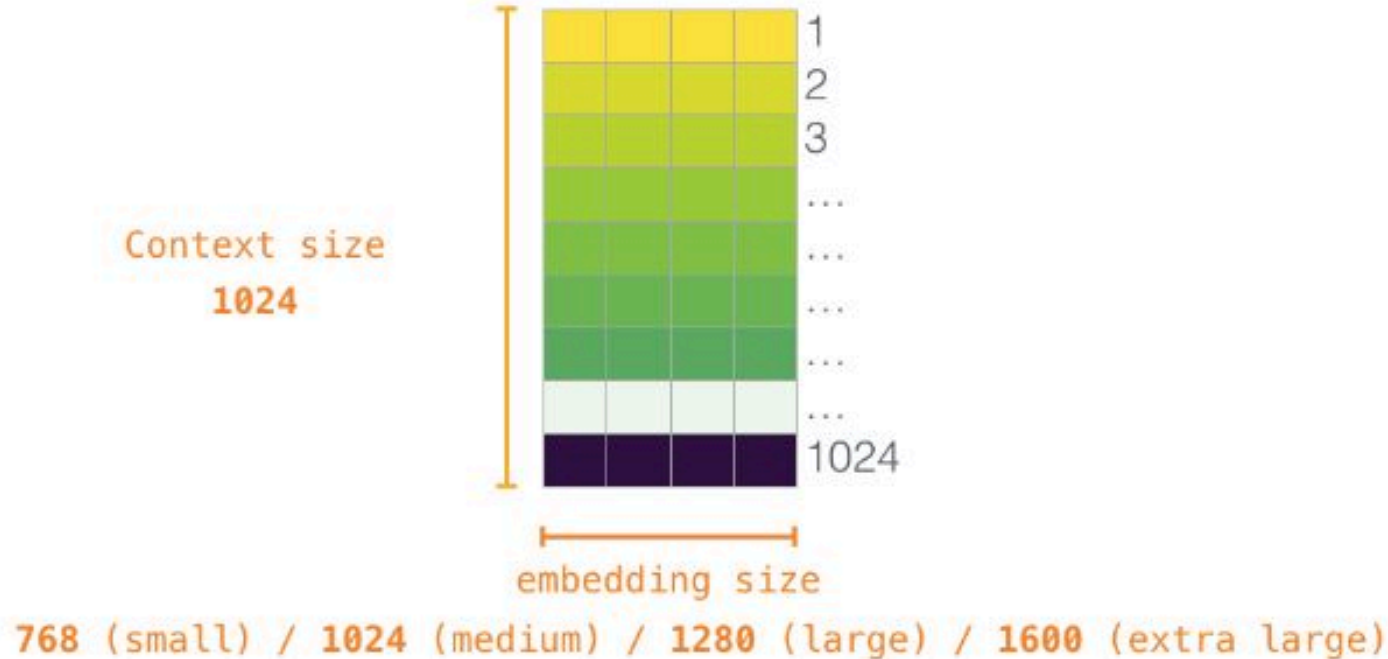
AUTOREGRESSIVE GENERATION

- **Token-by-Token:** The model begins with a start token (e.g., <S>) and iteratively predicts the next word one by one.
- **Vocabulary Scoring:** The final output vector is scored against the entire model vocabulary using a softmax layer to produce probabilities.
- **Sampling (Top-K):**
 - Instead of always choosing the highest probability (Max-prob/Argmax), the model samples from the top-K probable tokens.
 - This injects variety and creativity into the generated text.
- **Tokenization (BPE):** Uses Byte Pair Encoding to create subword tokens, balancing vocabulary size and out-of-vocabulary handling.



GPT Attention Mechanism & Process

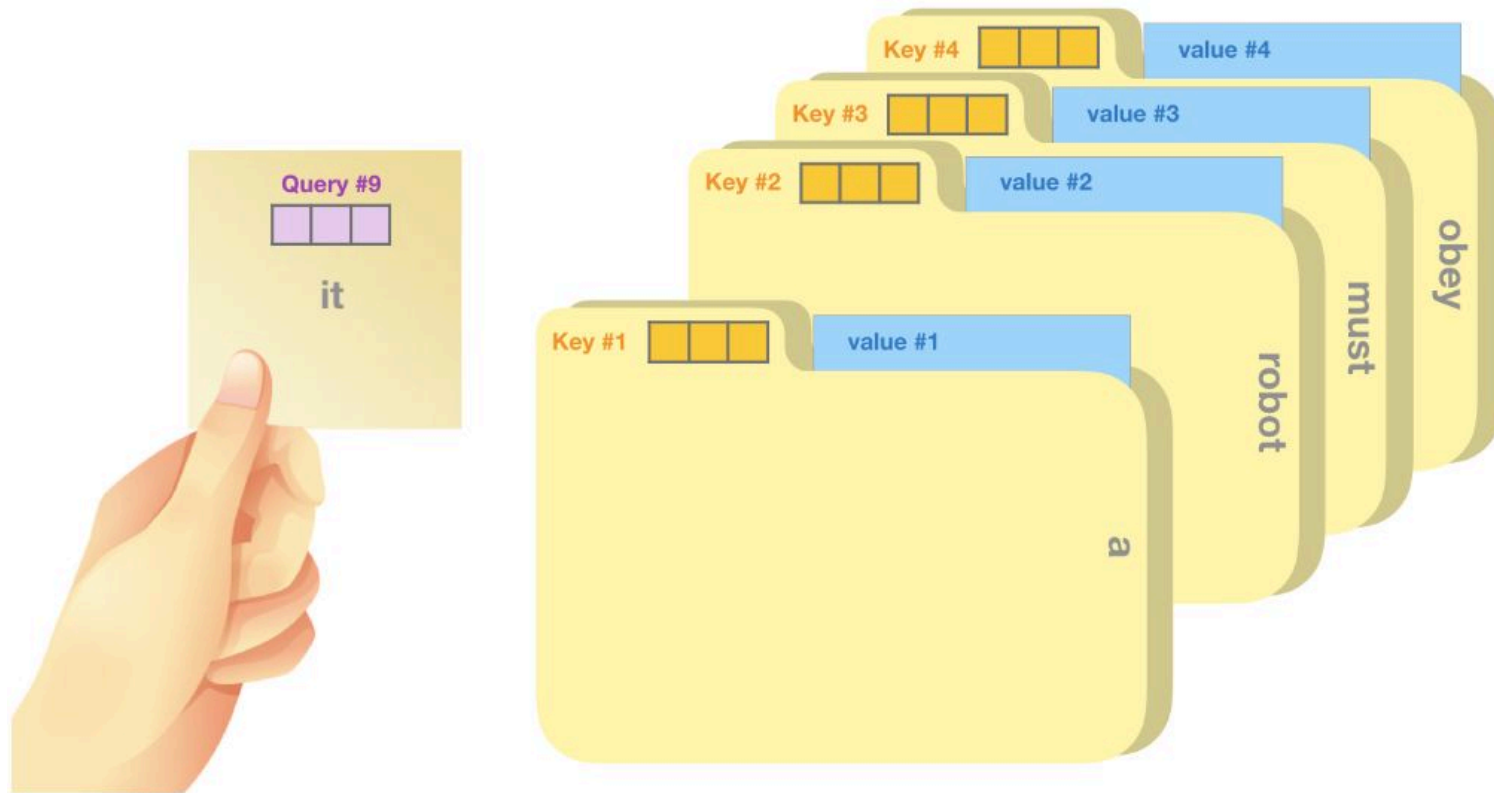
Positional Encodings (wpe)



Processing Details

- **Autoregressive Prediction:** The GPT model generates output sequentially, predicting exactly one token at a time based on preceding tokens.
- **Masked Self-Attention:** Employs a causal mask to ensure the model can only attend to past and present tokens, avoiding "peeking" into the future.
- **No Cross-Attention:** As a decoder-only model, it drops the cross-attention layer used in traditional encoder-decoder architectures.
- **Deep Context Processing:** Each token passes through multiple transformer blocks to build a rich, deeply contextualized representation.

GPT Self-Attention Details

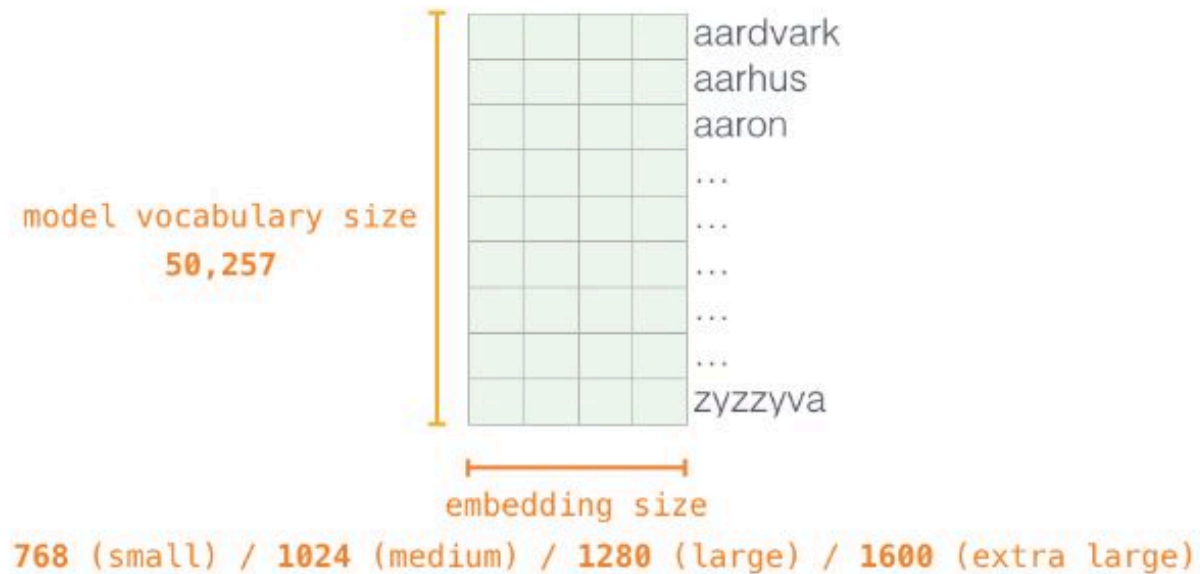


Causal Masking

- **Masked Self-Attention:** GPT relies heavily on causal masking to prevent the model from observing future tokens during generation.
- **Information Flow:** Unlike BERT, information flows strictly from left to right, maintaining the context needed for autoregressive tasks.
- **Attention Matrix:** The upper triangular part of the attention matrix is masked out before applying softmax, restricting access to unseen text.
- **Language Generation:** This approach allows GPT to effectively and coherently predict the next word one token at a time.

GPT Sampling and Generation Process

Token Embeddings (wte)



Each row is a word embedding: a list of numbers representing a word and capturing some of its meaning. The size of that list is different in different GPT2 model sizes. The smallest model uses an embedding size of 768 per word/token.



Generation Details

- **Autoregressive Output:** GPT models generate sequences token-by-token. Each new token is appended to the input context for the next prediction.
- **Vocabulary Scoring:** The final output vector is projected to the vocabulary size, generating logits that are converted to probabilities via softmax.
- **Top-K Sampling:** Instead of greedily picking the highest probability (Max-prob), GPT samples from the top K tokens to introduce diversity.
- **BPE Tokenization:** Uses Byte Pair Encoding to effectively handle subword representations, managing both common words and rare morphological fragments seamlessly.

ChatGPT Training Pipeline Overview



<USER>
Can you write a short introduction about the relevance of the term "monopsony" in economics? Please use examples related to potential monopsonies in the labour market and cite relevant research.

<ASSISTANT>
"Monopsony" refers to a market structure where there is only one buyer for a particular good or service. In economics, this term is particularly relevant in the labor market, where a monopsony employer has significant power over the wages and working conditions of their employees. The presence of a monopsony can result in lower wages and reduced employment opportunities for workers, as the employer has little incentive to increase wages or provide better working conditions...

every
~year

Stage 1: Pretraining

1. Download ~10TB of text.
2. Get a cluster of ~6,000 GPUs.
3. Compress the text into a neural network, pay ~\$2M, wait ~12 days.
4. Obtain **base model**.

every
~week

Stage 2: Finetuning

1. Write labeling instructions
2. Hire people (or use [scale.ai](#)!), collect 100K high quality ideal Q&A responses, and/or comparisons.
3. Finetune base model on this data, wait ~1 day.
4. Obtain **assistant model**.
5. Run a lot of evaluations.
6. Deploy.
7. Monitor, collect misbehaviors, go to step 1.



Training Stages

- **1. Pre-training:** The base model learns language structure and wide representations from a vast unsupervised dataset.
- **2. Supervised Fine-Tuning (SFT):** An assistant model is fine-tuned using high-quality, task-specific demonstrations and instructions.
- **3. Reward Modeling:** A secondary model is trained to score generation outputs based on human preference and feedback ratings.
- **4. RLHF:** Reinforcement learning (e.g., PPO) optimizes the primary language model to maximize the predicted human-aligned reward.

RLHF: Reinforcement Learning Training

Blue are the prompt tokens, identical across rows
Yellow are completion tokens by the model (initialized with SFT model)
Green is the special <|reward|> token "readout", RM now predicts these
Only the yellow cells are trained on, the rest are ignored.

The sampled tokens become labels, but the training objective is weighted by the "advantage" (normalized rewards)

In this example:

- Row #1 tokens were great. These get their probabilities boosted.
- Row #2 tokens were bad. These get their probabilities decreased.
- Row #3 tokens were ~ok. These get their probabilities slightly boosted.



RL Optimization

- **Policy Optimization:** Fine-tunes the language model using algorithms like Proximal Policy Optimization (PPO).
- **Reward Signal:** Employs the previously trained Reward Model to evaluate and score the generated text outputs.
- **Iterative Learning:** The model continuously generates responses, receives scores, and adjusts its internal weights to maximize expected rewards.
- **Alignment Goal:** The final objective is to align the assistant's behavior strictly with human preferences for safety and helpfulness.

Input Encoding and Tokenization

PREPARING INPUT FOR TRANSFORMERS

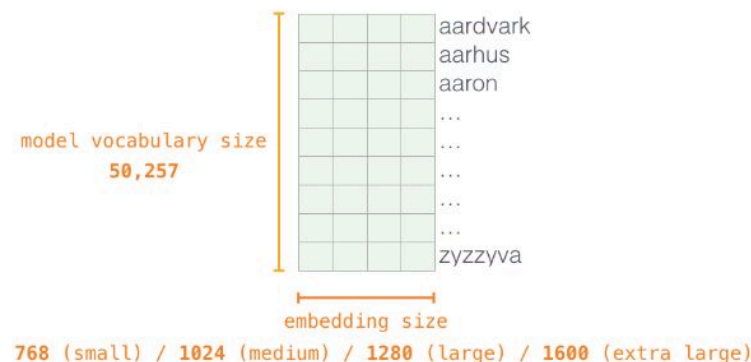
Before raw text can be processed by a Transformer, it must be converted into numerical representations. This process involves **tokenization**, creating continuous **embeddings**, and injecting sequence order via **positional encodings**.



Key Concepts

- **Byte Pair Encoding (BPE):** A subword tokenization strategy (used in GPT) that balances vocabulary size and flexibility by breaking rare words into frequent subword units.
- **Positional Encodings:** Since self-attention has no inherent notion of sequence order, fixed or learned positional vectors are added to the input embeddings.
- **K/V Caching:** An optimization technique during generation where previously computed Keys and Values are stored in memory to avoid redundant calculations.

Token Embeddings (wte)



Each row is a word embedding: a list of numbers representing a word and capturing some of its meaning. The size of that list is different in different GPT2 model sizes. The smallest model uses an embedding size of 768 per word/token.

Large Language Models (LLMs): Overview

WHAT IS AN LLM?

Large Language Models are trained on vast datasets using self-supervised learning techniques. They use language modeling to predict the next word in a sequence based on context, helping them understand structure, grammar, and meaning.



Key Components Influencing LLM Architecture

- **Model Size and Parameter Count:** Scales capability and capacity.
- **Input Representations:** How text is tokenized and embedded.
- **Self-Attention Mechanisms:** Allows the model to weigh context appropriately.
- **Training Objectives:** Tasks like next-token prediction or masked language modeling.
- **Computational Efficiency:** Techniques to handle large-scale training and inference.
- **Decoding and Output Generation:** Strategies for producing coherent and relevant text.



LLM Training Pipeline



STAGE 01
Pre-training

Creates the Base Model. Trained on massive, unsupervised web-scale datasets to build foundational language understanding.



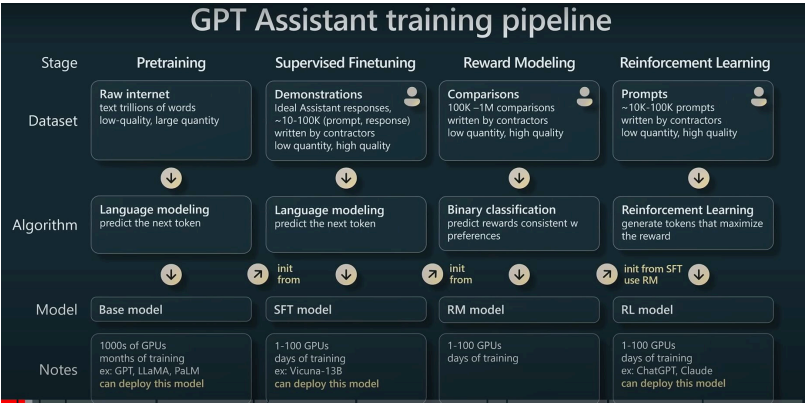
STAGE 02
Fine-tuning

Creates the Assistant Model. Adapts the pre-trained model for specific tasks or instructions using smaller, high-quality datasets.



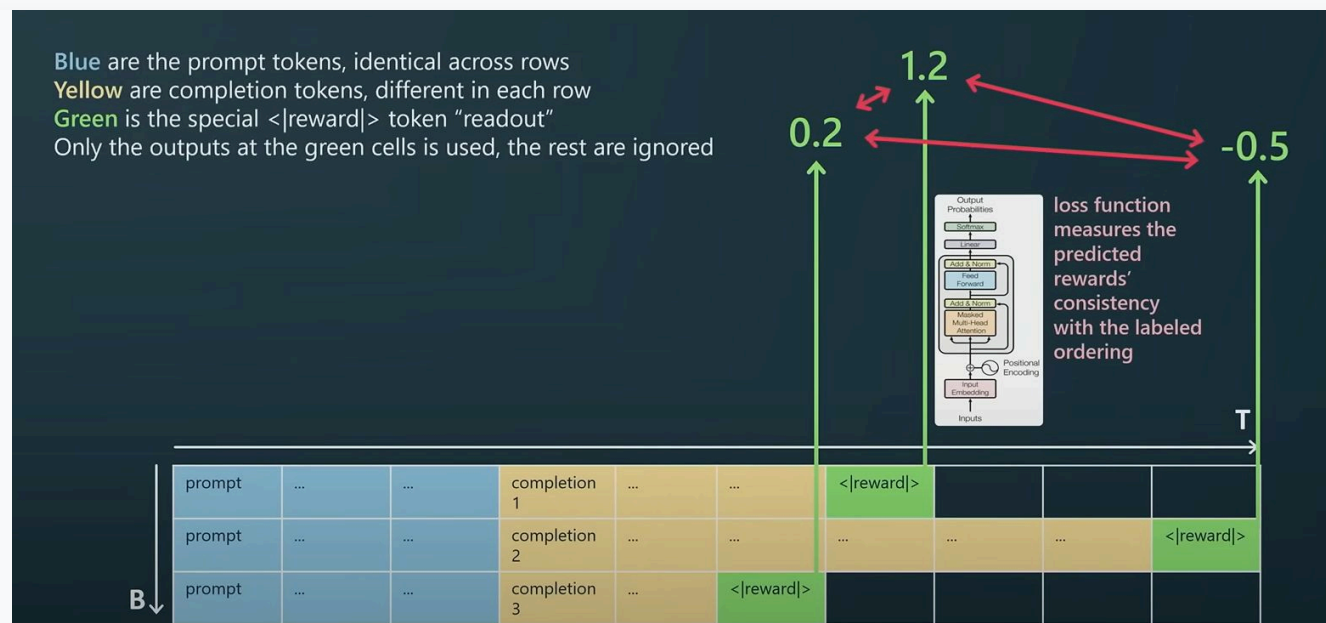
STAGE 03 (OPTIONAL)
RLHF

Reinforcement Learning from Human Feedback. Uses reward modeling to align model responses with human preferences.



RLHF: Reward Modeling and Reinforcement Learning

Aligning language models with human preferences using a multi-stage training pipeline.



01

Reward Modeling (RM)

Train a separate model on human comparisons to predict which response a human would prefer. It outputs a scalar reward score.

02

Reinforcement Learning

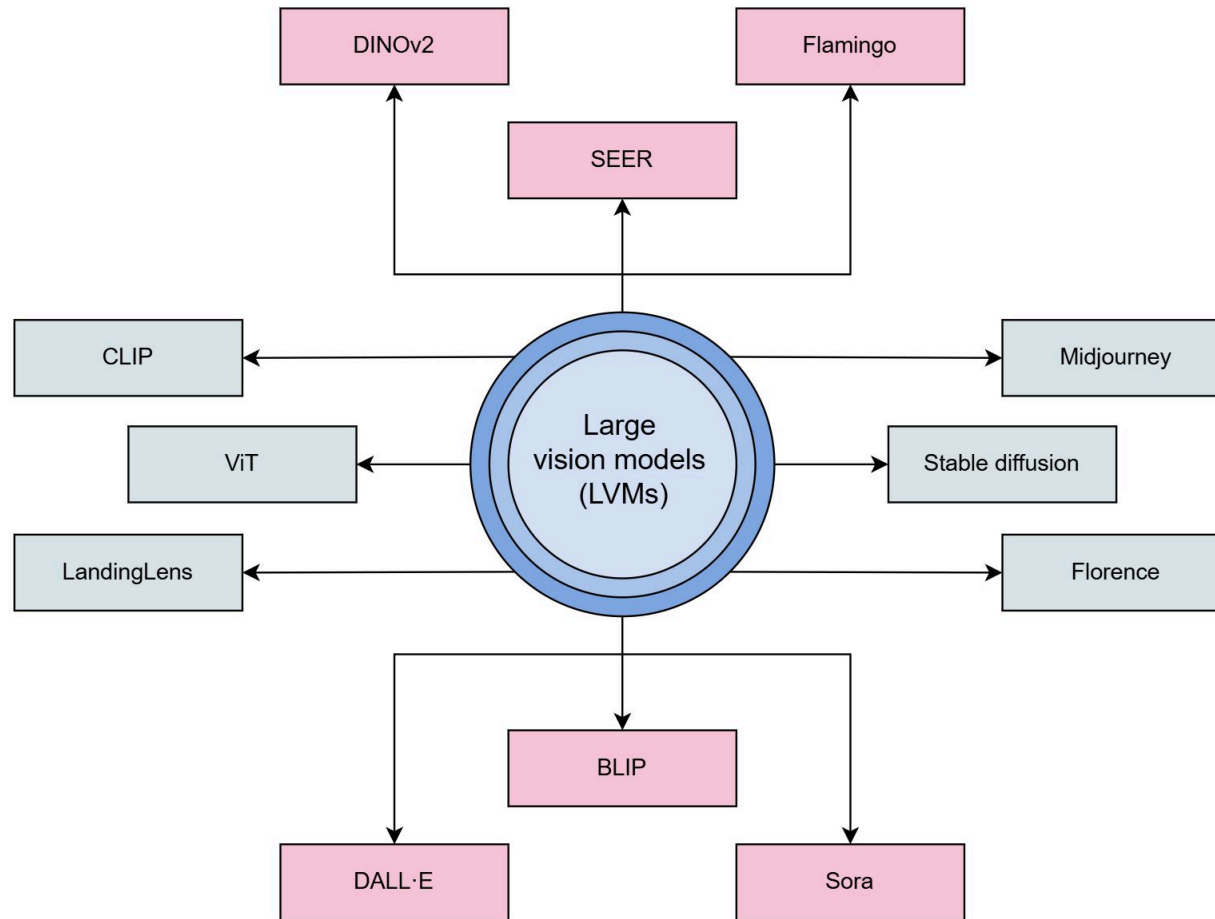
Use an RL algorithm (e.g., Proximal Policy Optimization - PPO) to fine-tune the LLM by maximizing the predicted reward.

03

Alignment & Outcome

The resulting assistant model handles ambiguity better, reduces harmful outputs, and is safer and more helpful to users.

Large Vision Models (LVMs): Examples



Prominent LVMs

- **ViT:** Google's Vision Transformer, introducing patch-based visual processing without standard convolutions.
- **CLIP:** OpenAI's Contrastive Language-Image Pretraining bridging text descriptions with visual concepts.
- **GPT-4o:** Advanced multimodal model processing text, audio, and vision seamlessly in a unified architecture.
- **DINO:** Self-distillation framework capable of producing high-quality visual features without needing human labels.

Evolution of Large Vision Models

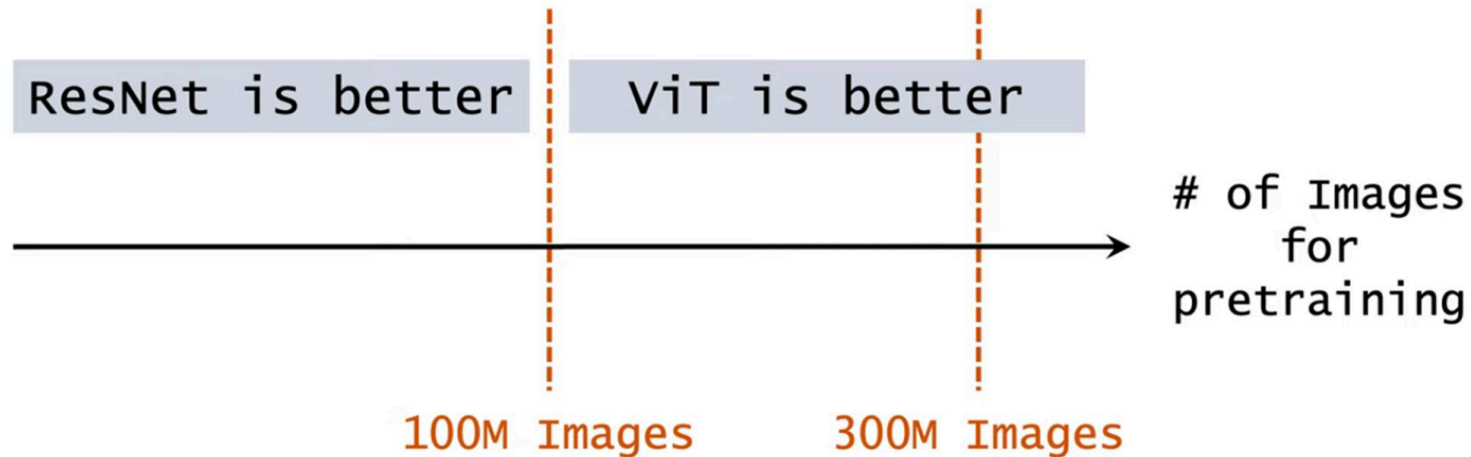
Early 2010s

The rise of convolutional neural networks (CNNs) for image classification tasks, exemplified by AlexNet's breakthrough performance in the 2012 ImageNet competition.

Key Milestones

- **CNN Era:** Computer vision tasks were long dominated by robust Convolutional Neural Networks.
- **ViT Breakthrough:** Adapting the standard Transformer architecture directly to image patches revolutionized visual modeling.
- **Scaling Data:** ViTs began to heavily outperform CNNs when pre-trained on massive datasets (100M+ images).
- **Self-Supervision:** Techniques like DINO enabled learning rich, intricate visual representations without human labels.
- **Multimodal Era:** Models like CLIP bridged the gap, learning transferable representations from language and vision together.

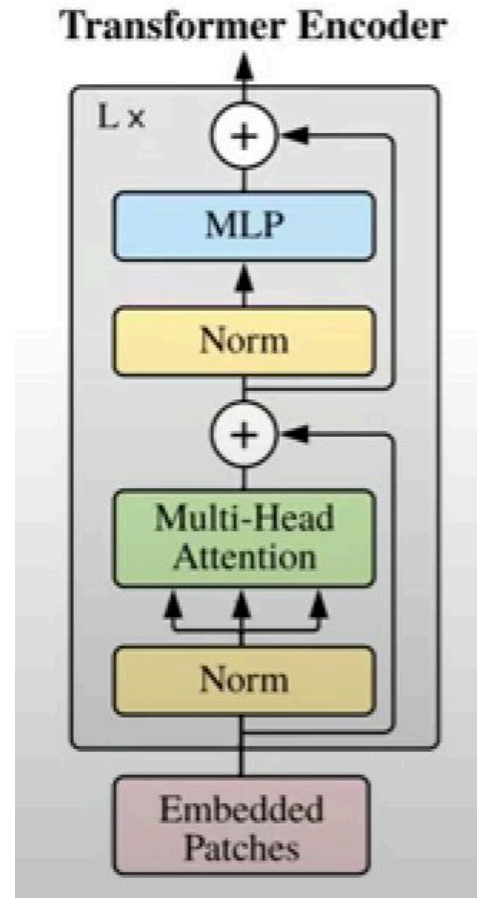
Vision Transformer (ViT) Architecture Overview



ViT Motivation & Architecture

- **Beyond CNNs:** While CNNs like ResNet previously dominated image classification, ViT offers a new paradigm.
- **Patch Processing:** Images are split into fixed-size patches (e.g., 16x16), linearly embedded, and treated identically to word tokens in NLP.
- **Large-Scale Advantage:** ViT successfully beats CNNs when the pre-training dataset is sufficiently large (at least 100 million images).
- **Standard Transformer:** Relies almost entirely on the standard Transformer encoder framework without image-specific inductive biases.

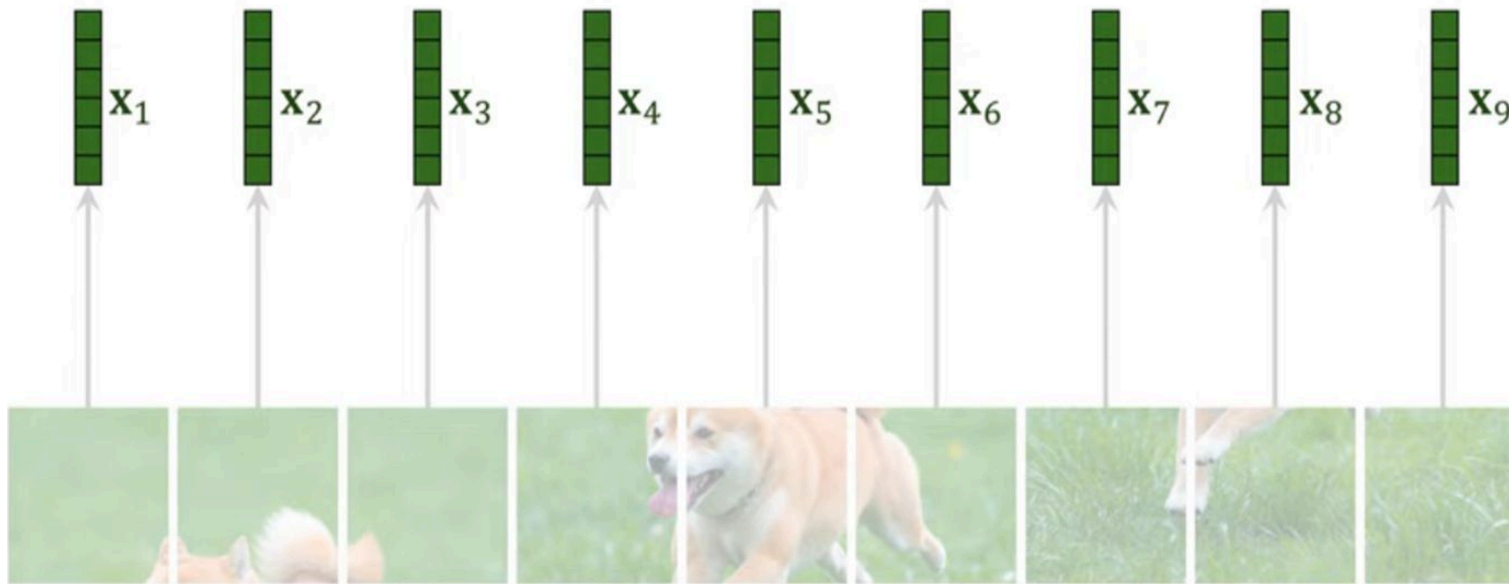
Vision Transformer (ViT) Encoder Architecture



Encoder Details

- **Alternating Layers:** Consists of alternating Multi-head Self-Attention (MSA) and Multi-Layer Perceptron (MLP) blocks.
- **Pre-Norm Strategy:** Layer Normalization (LN) is applied *before* every block, which differs from the original post-norm transformer.
- **Skip Connections:** Residual connections are added *after* every block to ensure smooth gradient flow during training.
- **NLP Inspiration:** The architecture heavily borrows from the standard NLP Transformer encoder (Vaswani et al., 2017).

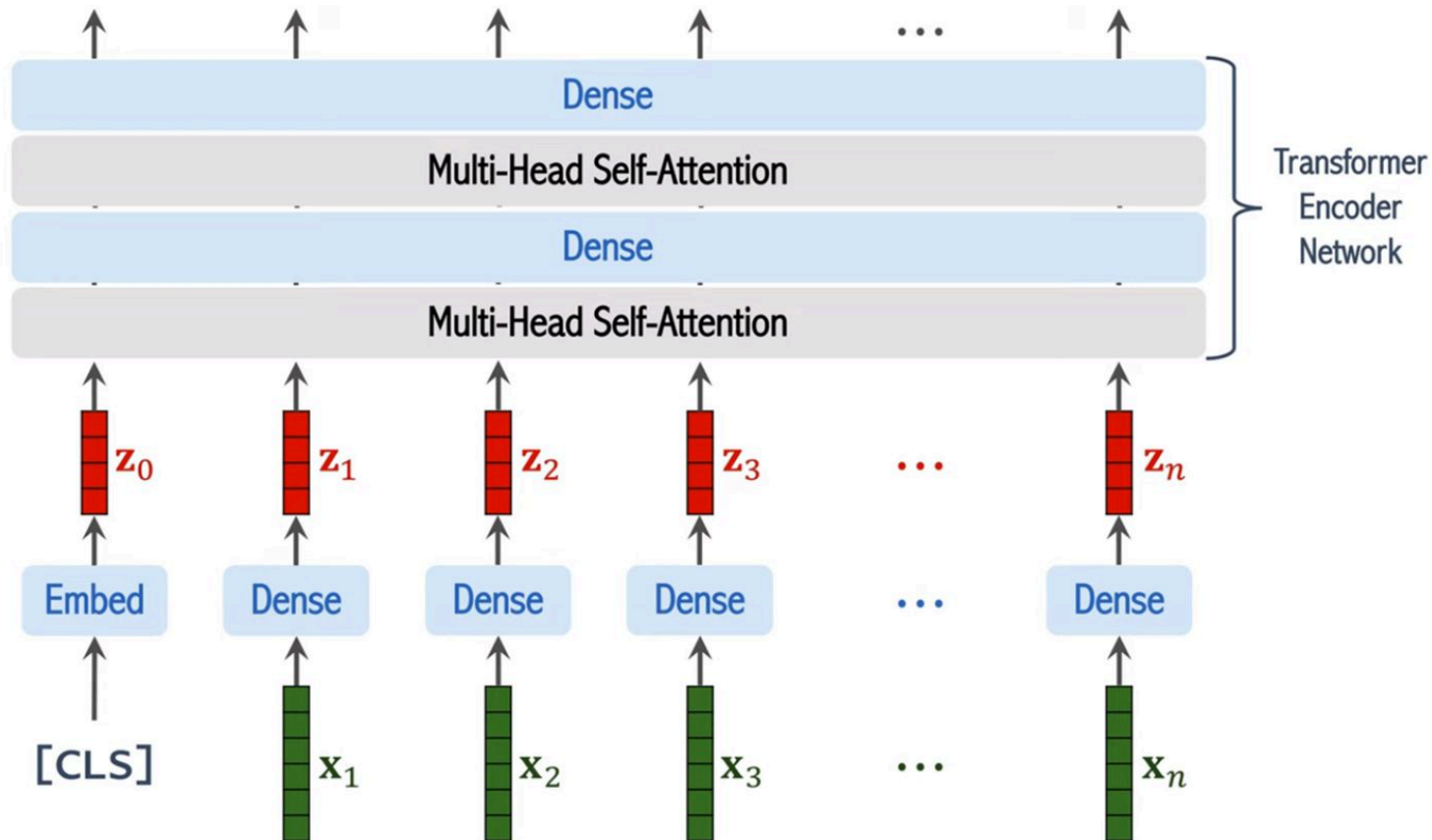
ViT Training: Patch Splitting & Flattening



Patch Processing

- **Grid Division:** The input image is split into a uniform grid of fixed-size, non-overlapping patches (e.g., 16x16 pixels).
- **Patch Flattening:** Each 2D patch of shape (h, w, c) is unrolled into a flat 1D vector of length $(h \cdot w \cdot c)$.
- **Linear Projection:** The flattened vectors are mapped to a lower-dimensional embedding space using a trainable linear layer.
- **Sequence Format:** This generates a sequence of token embeddings, mirroring how words are fed into an NLP Transformer.

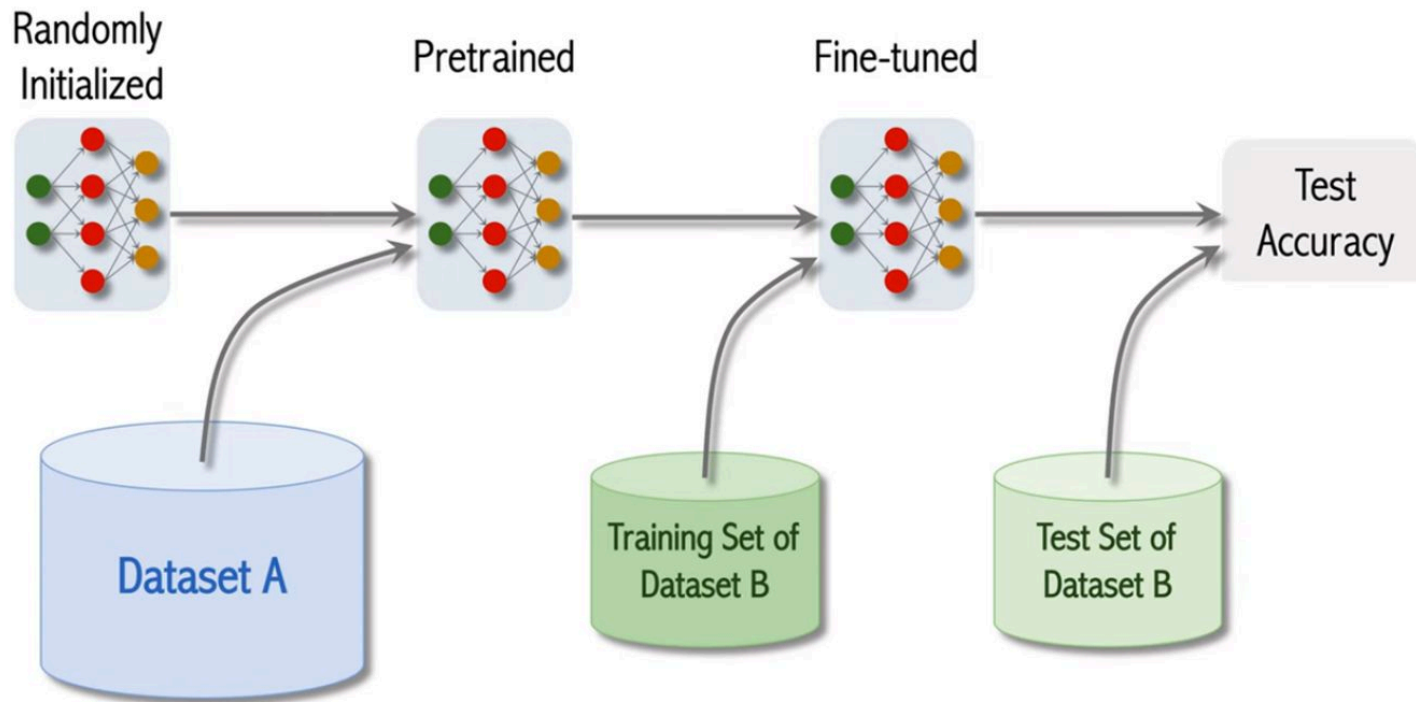
ViT CLS Token Mechanism



The [CLS] Token

- **Initialization:** A randomly initialized vector prepended to the sequence of embedded image patches.
- **Learned Embeddings:** Its representation is learned dynamically alongside the patch embeddings during training.
- **Global Representation:** The state of the [CLS] token at the final output of the Transformer encoder serves as the aggregate image representation.
- **Classification Head:** The final classification MLP is exclusively attached to this single token's output to perform the downstream task.

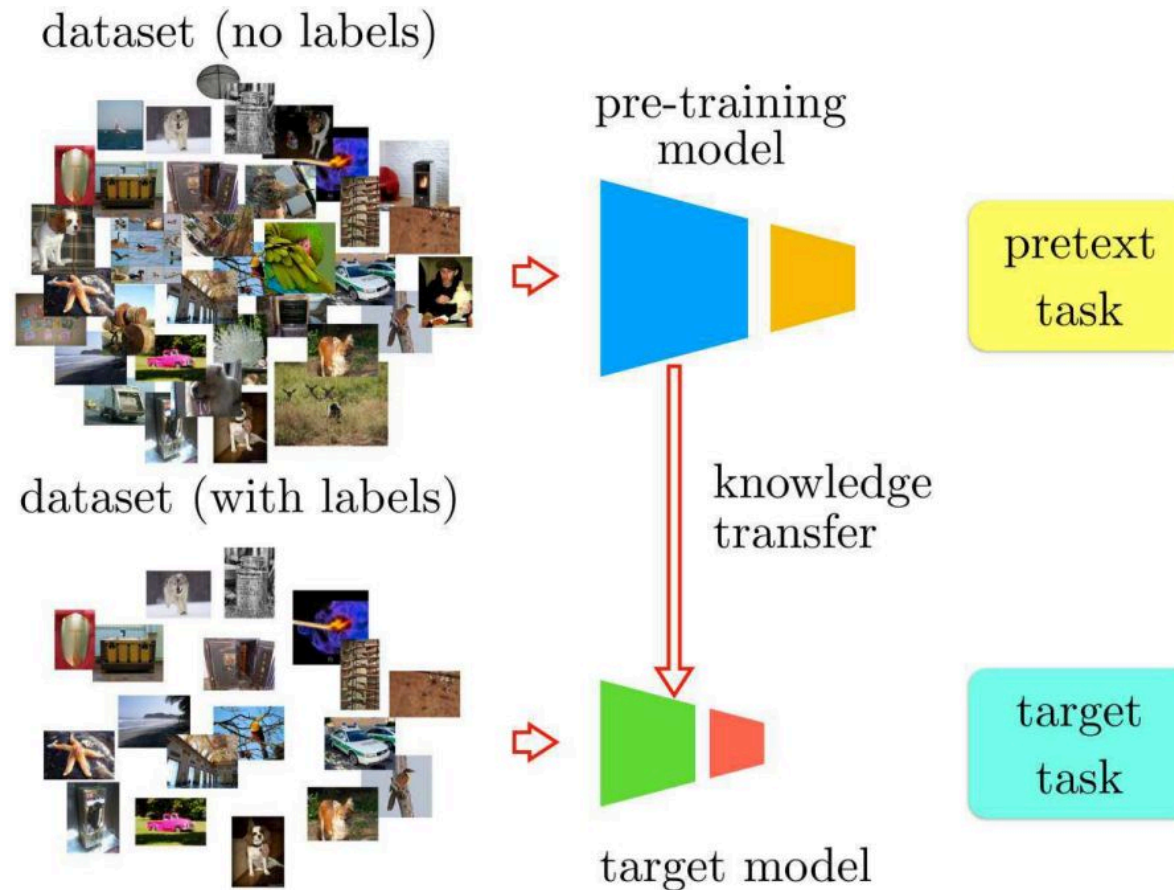
Vision Transformer (ViT) Training Pipeline



Complete Training

- **Initialization:** Start by randomly initializing the Vision Transformer model parameters.
- **Pre-training:** Train on a massive, general-purpose image dataset (Dataset A) to learn fundamental visual features and robust representations.
- **Fine-tuning:** Adapt the pre-trained model to a specific downstream task using a smaller, high-quality, and specialized training dataset.
- **Evaluation:** Assess the model's final performance and accuracy on the unseen test set of the target dataset (Dataset B).

DINO: Self-Distillation with No Labels



Self-Supervised ViT

- **Teacher-Student Setup:** Employs two networks (online student and momentum teacher) interacting and learning from each other.
- **No Labels Needed:** Relies on self-supervised learning where the model generates its own pseudo-labels for pretext tasks.
- **BYOL Inspiration:** Deeply inspired by "Bootstrap Your Own Latent" architecture adapted for visual recognition.
- **Emergent Properties:** Capable of learning robust intermediate representations and clear object boundaries without any human annotation.

Large Vision Models (LVMs)

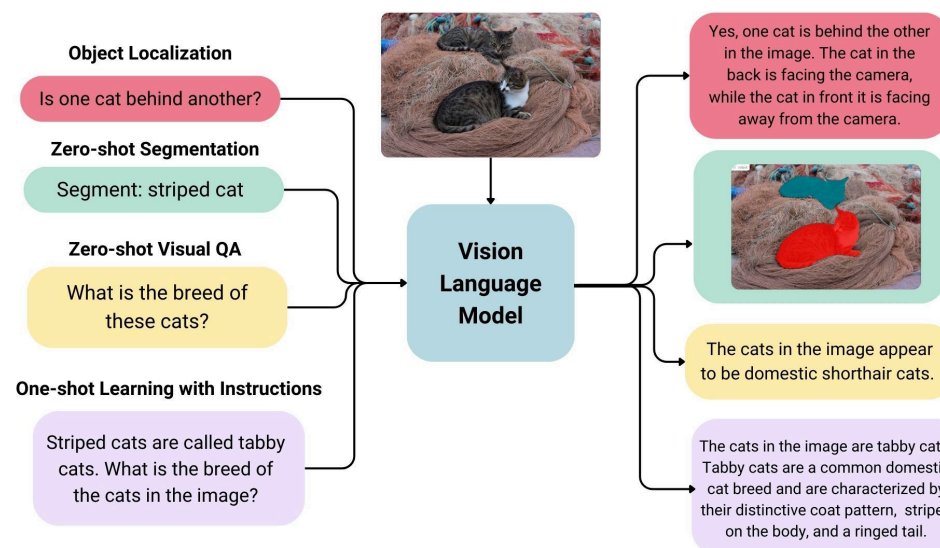
INTRODUCTION TO LVMS

Large Vision Models (LVMs) are advanced, **transformer-based architectures** designed for complex visual tasks. They are typically **pre-trained on massive datasets** (e.g., ImageNet) and fine-tuned for specialized applications like object detection and segmentation.



Prominent Examples

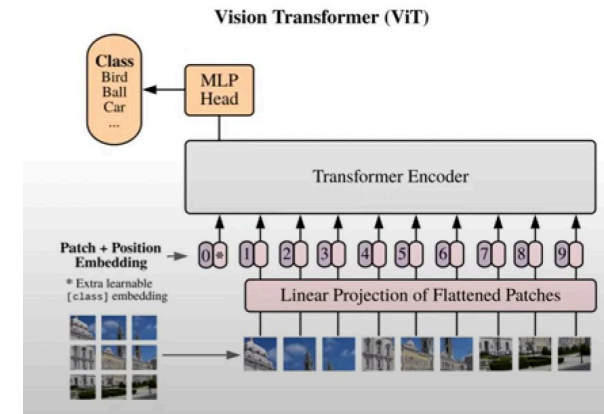
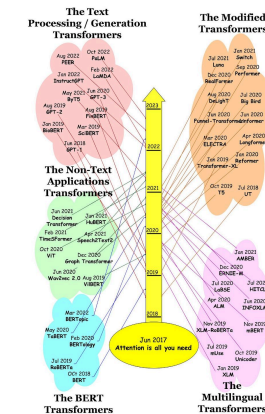
- **Vision Transformer (ViT)**: Google's pioneering model adapting transformer encoders to image patches.
- **CLIP (Language-Image Pretraining)**: OpenAI's multimodal model bridging text and images.
- **GPT-4o**: OpenAI's omni-model seamlessly integrating advanced vision capabilities.
- **DINO**: A self-supervised method (self-distillation with no labels) for learning robust visual features.



Vision Transformer (ViT): Motivation and Overview

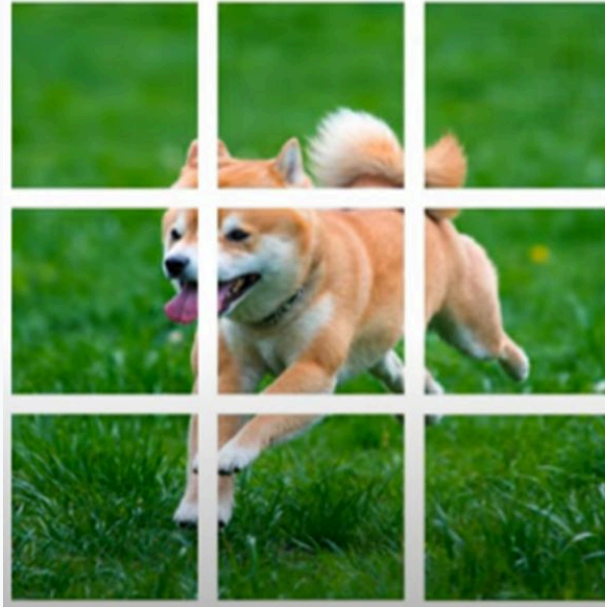
WHY TRANSFORMERS FOR VISION?

- **Adapting NLP Success:** In natural language processing, the Transformer architecture became the dominant paradigm.
- **Challenging CNNs:** Historically, Convolutional Neural Networks (CNNs) like ResNet dominated computer vision modeling.
- **Data Scale Dependency:** ViT demonstrates that Transformers can outperform CNNs, but only when the pre-training dataset is sufficiently large (at least 100 million images).
- **Encoder-Only Architecture:** Employs an encoder-only Transformer architecture applied directly to sequences of image patches.



ViT as a Classifier: End-to-End Flow

Transforming an image into a sequence of tokens for standard Transformer encoder processing.



01

Patch & Flatten

Split the original image into fixed-size grids (e.g., 16x16 patches) and flatten them from 3D volumes into 1D vectors.

02

Linear Projection

Linearly project flattened patches into lower-dimensional space. Add positional encodings and prepend the [CLS] token.

03

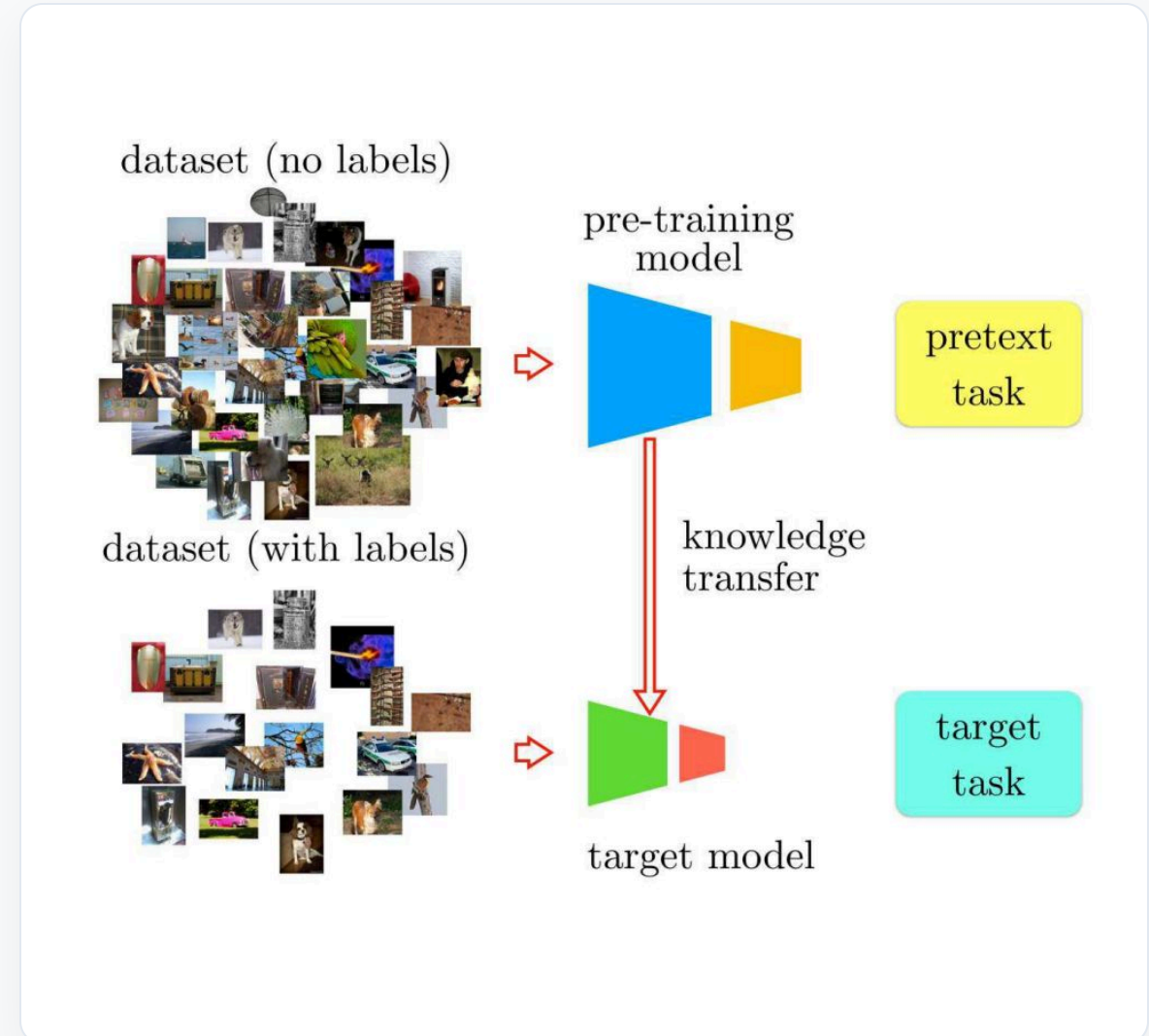
Transformer & Head

Process sequence through Transformer blocks. Use the final state of the [CLS] token for the classification head.

DINO: Self-Distillation with No Labels

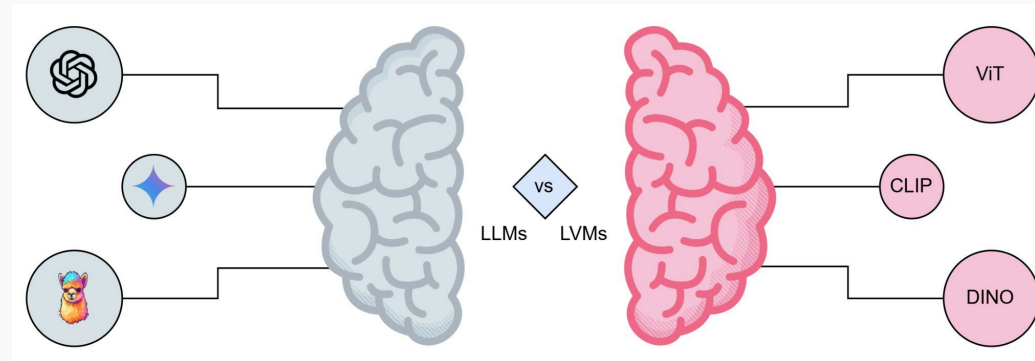
SELF-SUPERVISED VISION

- **Self-Supervised Pre-training:** Guides the model to learn intermediate representations without human annotations via pretext objectives.
- **Inspired by BYOL:**
 - Uses two interacting neural networks.
 - Teacher network (online).
 - Student network (target).
- **Knowledge Distillation:** The student network learns to predict the output of the dynamically updated teacher network.
- Emerging properties in **Vision Transformers (ViT)** features are observed through this self-supervised approach.



LLMs vs LVMs and Multimodal Convergence

Convergence: Combining language and vision through cross-modal transfer learning to improve performance on both types of tasks.



LLMs vs LVMs

- **Large Language Models (LLMs):** Understand, generate, and manipulate human language (e.g., GPT-4, PaLM 2, Claude).
- **Large Vision Models (LVMs):** Specialize in processing and understanding visual information.
- Historically separate domains with distinct architectures (Transformers for NLP, CNNs for Vision).
- Now unifying under **Transformer-based architectures** for both modalities.

AND



Multimodal Applications

- **CLIP:** Contrastive Language-Image Pre-training aligns images and text descriptions.
- **Visual Question Answering (VQA):** Answer questions about images combining language and vision.
- **Image Generation:** DALL·E generating images from text descriptions.
- **Video Understanding:** Analyzing video content integrating temporal, visual, and auditory cues.

Summary: Key Takeaways



The Transformer Core

- **Architecture:** Relies purely on attention mechanisms, eliminating recurrence and convolution.
- **Key Layers:** Built upon self-attention, positional encoding, layer normalization, and feed-forward networks.



Language Models: BERT vs GPT

- **BERT (Encoder):** Generates bidirectional context using Masked Language Modeling (MLM) and NSP.
- **GPT (Decoder):** Utilizes causal (masked) attention for autoregressive, unidirectional text generation.



The LLM Training Pipeline

- **Pre-training:** Massive self-supervised learning on vast corpora to build foundational knowledge.
- **Fine-tuning & RLHF:** Adapting to specific tasks and aligning with human preferences for safe, helpful outputs.



Large Vision Models (LVMs)

- **Vision Transformers (ViT):** Successfully processes images by treating them as sequences of distinct patches.
- **DINO & Self-Supervision:** Demonstrates powerful representation learning without human-annotated labels.

Future Directions

THE PATH FORWARD

The evolution of Transformer architectures is continuous. The focus is shifting towards **greater efficiency, broader modalities**, and **safer, more aligned models** as AI systems become deeply integrated into diverse applications.



Key Areas of Research and Development

- **Efficient Attention & Scaling Laws:** Reducing $O(n^2)$ complexity for longer contexts and optimizing resource utilization as models scale.
- **Better Alignment & Safety:** Moving beyond basic RLHF to more robust alignment techniques, ensuring helpful, harmless, and honest outputs.
- **Multimodal Reasoning:** Seamlessly integrating text, image, audio, and video processing into unified foundational models.
- **Self-Supervised Advances:** Pushing the boundaries of learning without human labels in both language and vision domains (e.g., next-gen DINO).